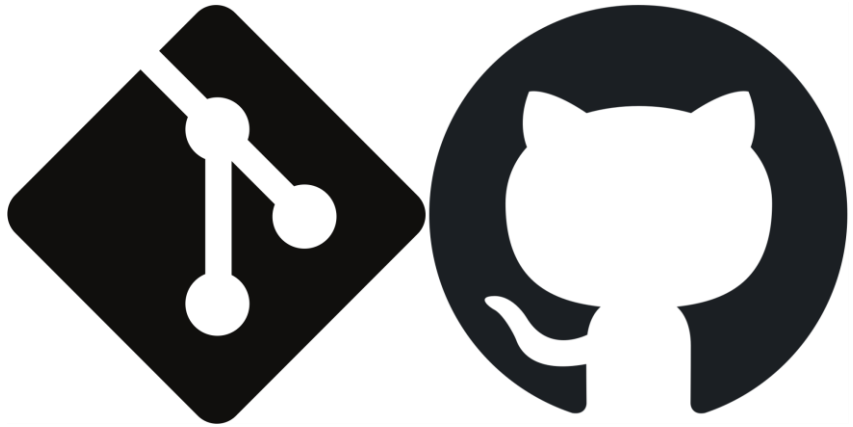


Introduction to Git & GitHub

Jennifer Graham

SAMS Session 3: Using GitHub to collaborate

12th May 2026

Topics to cover today

- Publishing code
  - Releases & DOIs
  - README & Wiki documentation
- GitHub tools
  - Issues
  - Projects
- Collaborative code development
  - Permissions, forking vs branching?
    - Pull requests?
  - GitHub Organisations

Recap from Sessions 1 & 2 …

So, what are Git & GitHub?

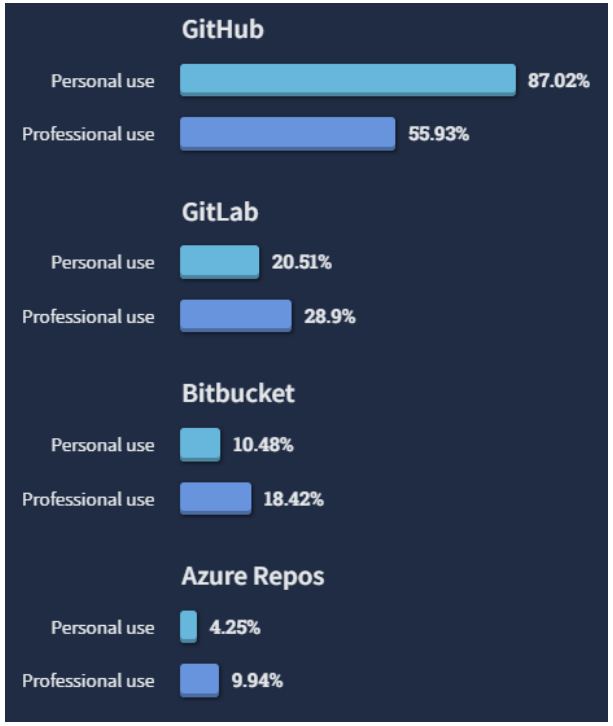
NB. These are <u>two different things</u>

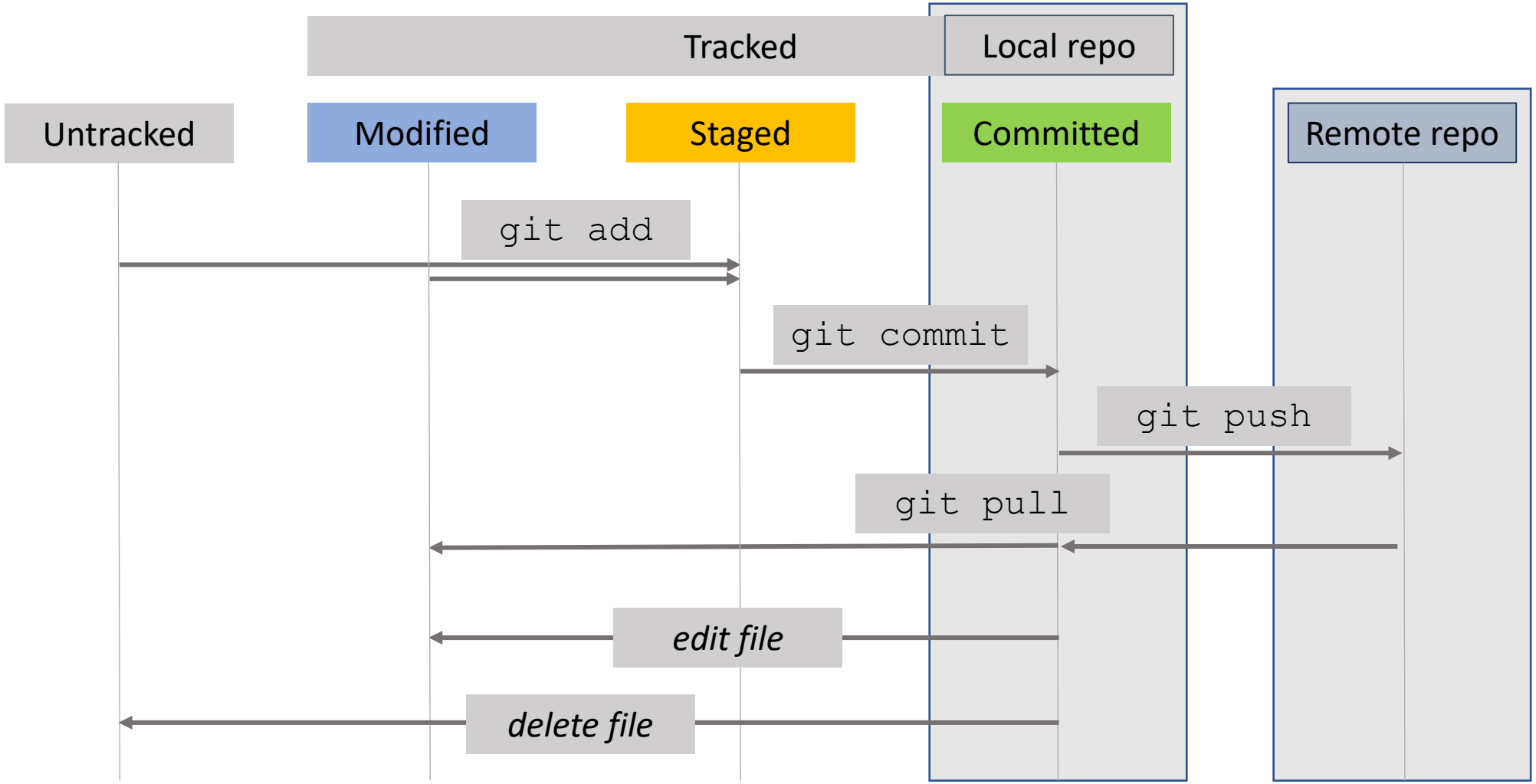
- Git = version control software.
- GitHub = web-based repository hosting service*.

i.e. Git is the software behind the GitHub web service.

You can use Git without GitHub. However, GitHub provides useful tools (especially for sharing your code).

* Other providers exist, but GitHub is the most popular, e.g.:



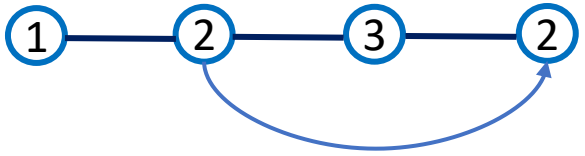

Tracked
Local repo
Untracked
Modified
Staged
Committed
Remote repo
git add
git commit
git push
git pull
edit file
delete file

```
git init                :: turn current directory into git repository.
git add <file>          :: stage the file for commit i.e., track changes.
git commit              :: confirm changes with message/explanation.

git push                :: publish/back-up changes on GitHub (remote server).
git pull                :: update your local repository with remote changes,
                               from GitHub (remote server).


git status              :: show which files have been tracked or modified.
git diff <file>         :: show difference between current file and last commit.
git log <file>          :: show commit history [for file]
```

How to roll back changes?

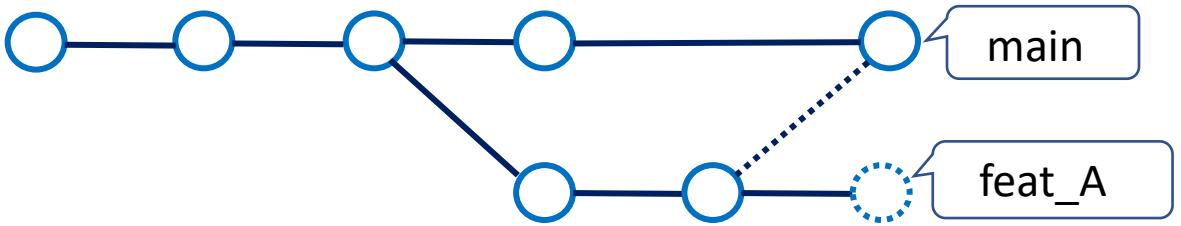
```
git restore <path>
:: discard changes in working directory for chosen file(s).

git checkout <commit> [--] <path>
:: retrieve version of file(s) from chosen commit.

git revert <commit>
:: undo your chosen commit, keeping record of change.

git reset [--soft/hard] <commit>
:: discard all commits [and possibly changes!] since chosen version
      - use with care!!!
```

git revert commit3



git reset --soft/hard commit2

Branches: How to create and merge?



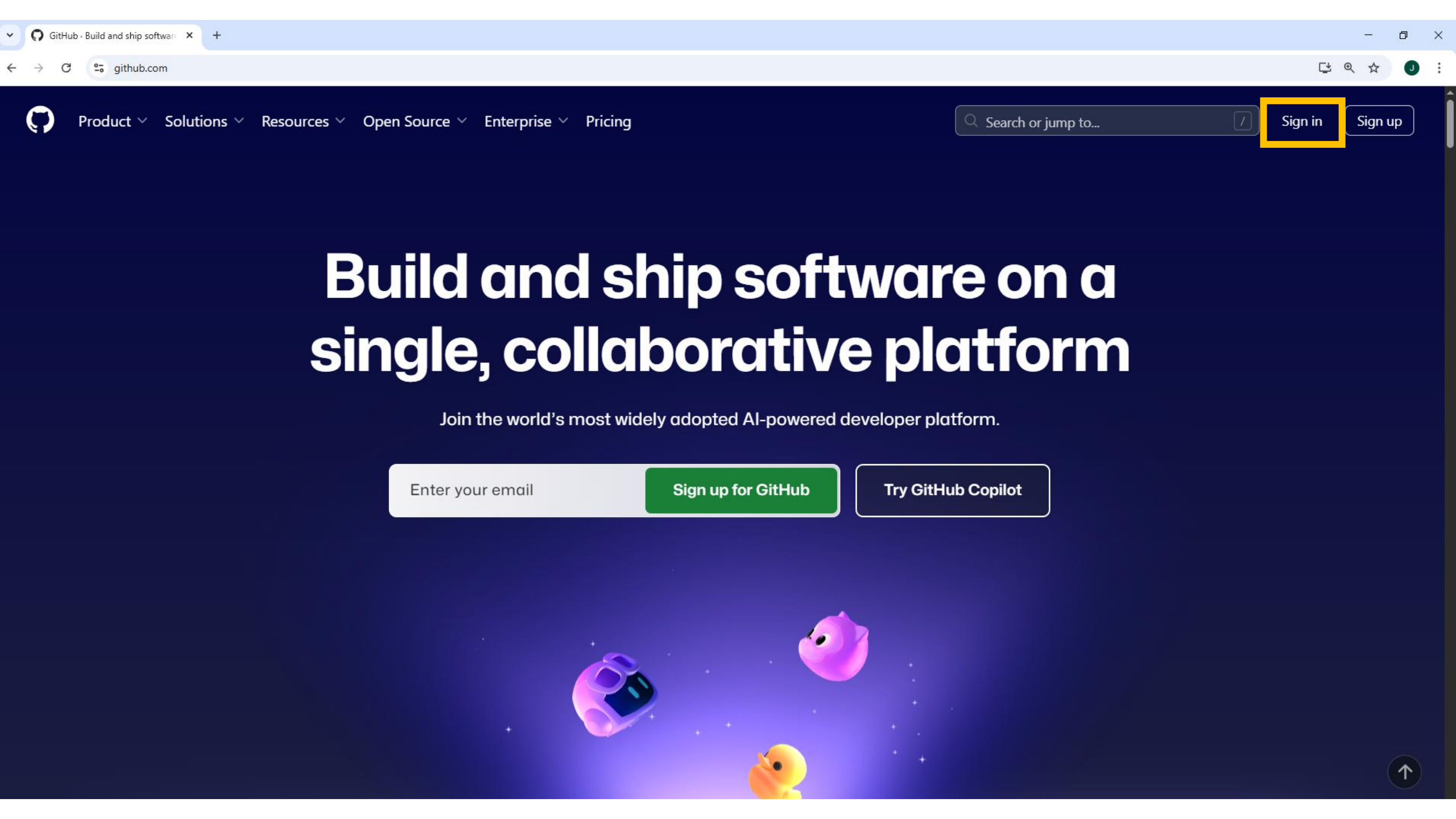

```
git branch  :: list existing branches; * = you are here.

git branch <branch-name>          :: create a new branch, <branch-name>.
git checkout <branch-name> :: "switch" onto branch, <branch-name>.

git switch [-c] <branch-name>
      :: switch to your branch, <branch-name>,
      -c option = create if new (shortcut for git branch and checkout).

git merge <branch-name>
      :: apply new commits from <branch-name> (since split), onto present branch

git diff <branch1>..<branch2> [<path>]
      :: check difference between two branches, for optional file(s) <path>
```

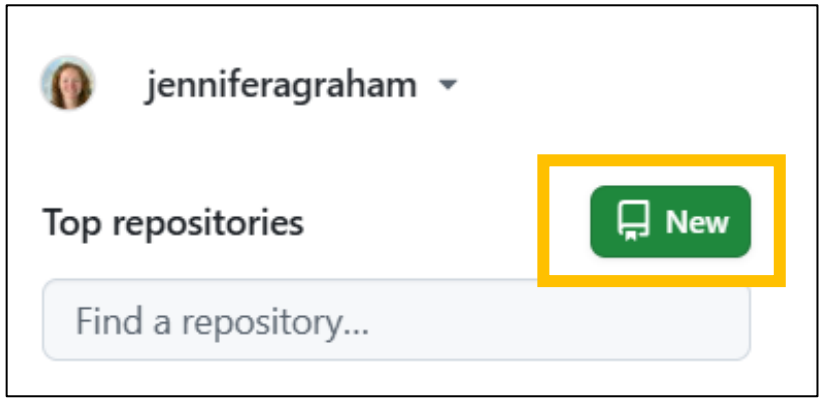
Publishing code on GitHub

Releases & DOIs

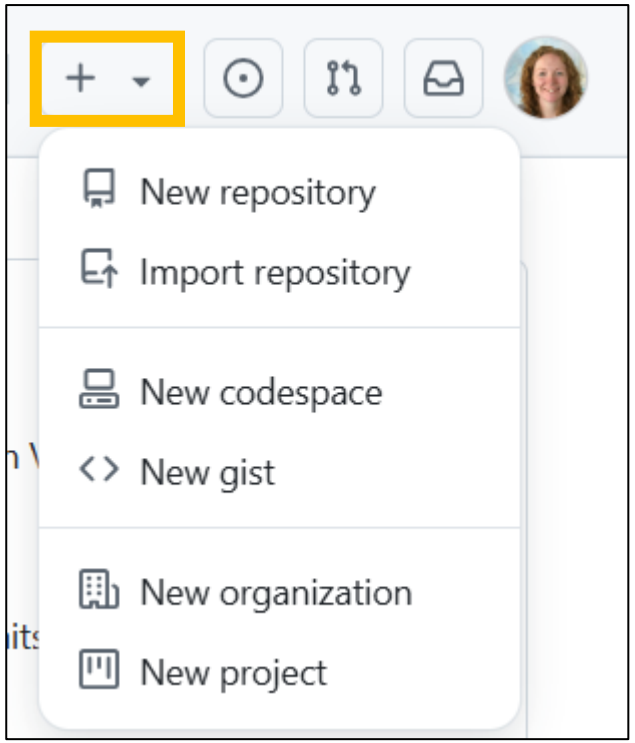
README

Wiki


GitHub · Build and ship softwar
github.com
Product
Solutions
Resources
Open Source
Enterprise
Pricing
Search or jump to...
/
Sign in
Sign up
Build and ship software on a single, collaborative platform
Join the world's most widely adopted AI-powered developer platform.
Enter your email
Sign up for GitHub
Try GitHub Copilot



Options to create your new repository.



New repository

https://github.com/new

Chat

New repository

Type / to search

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

1 **General**

Owner *

jenniferagraham /

Repository name *

hello_world

hello_world is available.

Great repository names are short and memorable. How about didactic-memory?

Description

0 / 350 characters

2 **Configuration**

Choose visibility *

Choose who can see and commit to this repository

Public

Public

Anyone on the internet can see this repository. You choose who can commit.

Private

You choose who can see and commit to this repository.

Add README

READMEs can be used as longer descriptions. About READMEs

Add .gitignore

.gitignore tells git which files not to track. About ignoring files

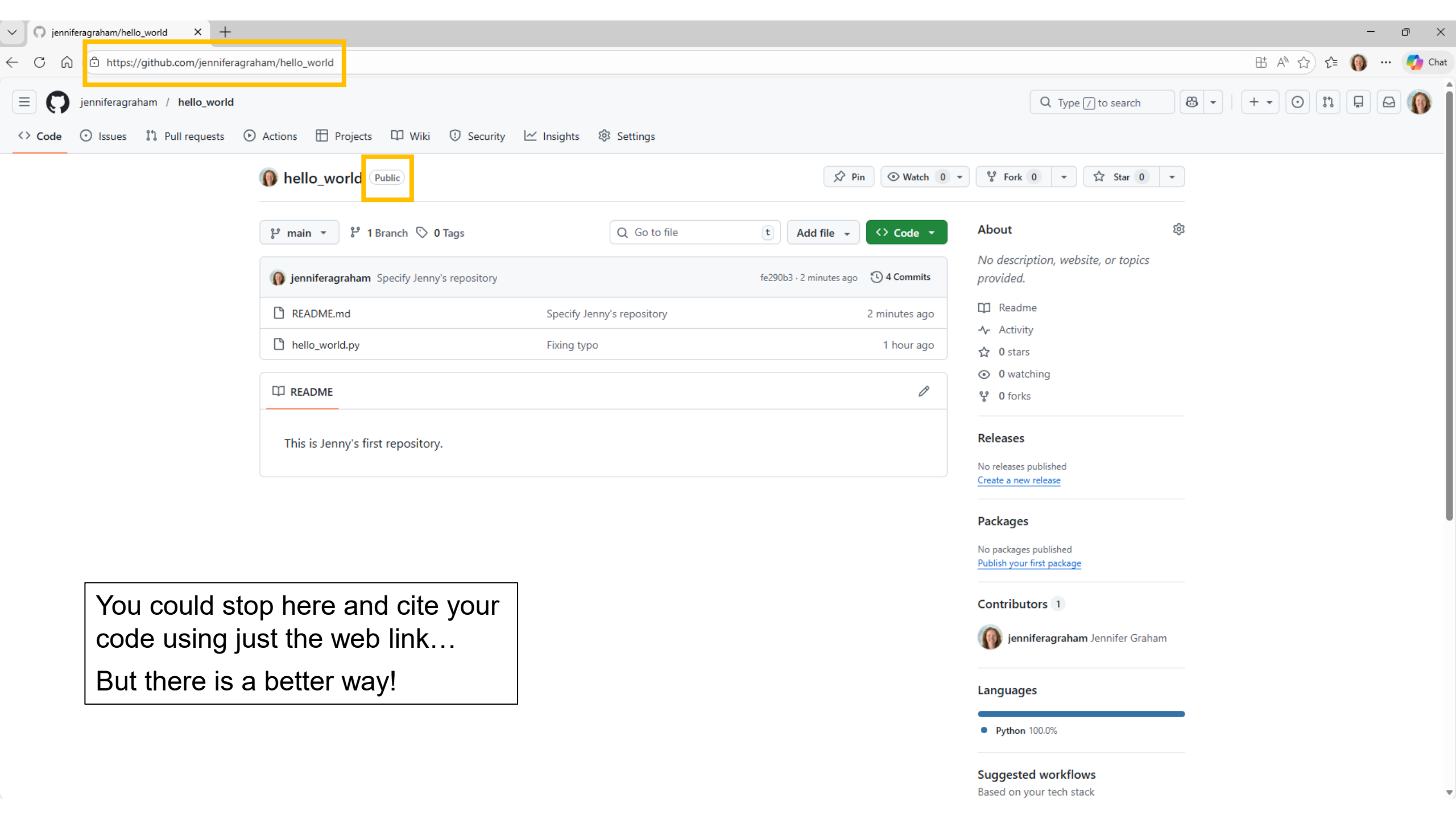
Add license

Licenses explain how others can use your code. About licenses

No license

Create repository

NB. GitHub's default is public, but you may want to ensure control over visibility.


jenniferagraham/hello_world
https://github.com/jenniferagraham/hello_world
Chat
jenniferagraham / hello_world
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
hello_world
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
t
Add file
Code
jenniferagraham Specify Jenny's repository
fe290b3 · 2 minutes ago
4 Commits
README.md
Specify Jenny's repository
2 minutes ago
hello_world.py
Fixing typo
1 hour ago
README
This is Jenny's first repository.
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
jenniferagraham Jennifer Graham
Languages
Python 100.0%
Suggested workflows
Based on your tech stack
You could stop here and cite your code using just the web link…
But there is a better way!

Before you publish…

Adding Documentation:

README

Wiki

Adding documentation

There are many ways you can provide documentation on what your code does and how someone else can use it.

- **README**
  - Most useful as it is automatically displayed on the front page of your repository.
  - Markdown formatting can be used
  - NB. Make sure to refer to any license (or restrictions) for use.
- **Wiki**
  - Broader information on use of repository.
  - Only available within public repositories or organisations.
  - e.g. https://github.com/jenniferagraham/Git_Training/wiki/
- **Issues**
  - Can flag development stages or known bugs/problems.
- **Projects**
  - Organise ongoing or planned work (related issues) on repositories?

NB. Your commit messages are still an important way to document the code development.

- NB. Some repository features are only available within organisations or paid accounts
  - More info here: https://github.com/pricing

README files

GitHub will automatically display your README on the front page of the repository.

- Markdown formatting can be used.
- Can be a simple text file, or any other text language?
- You can have new README files in each subdirectory.

Wiki pages

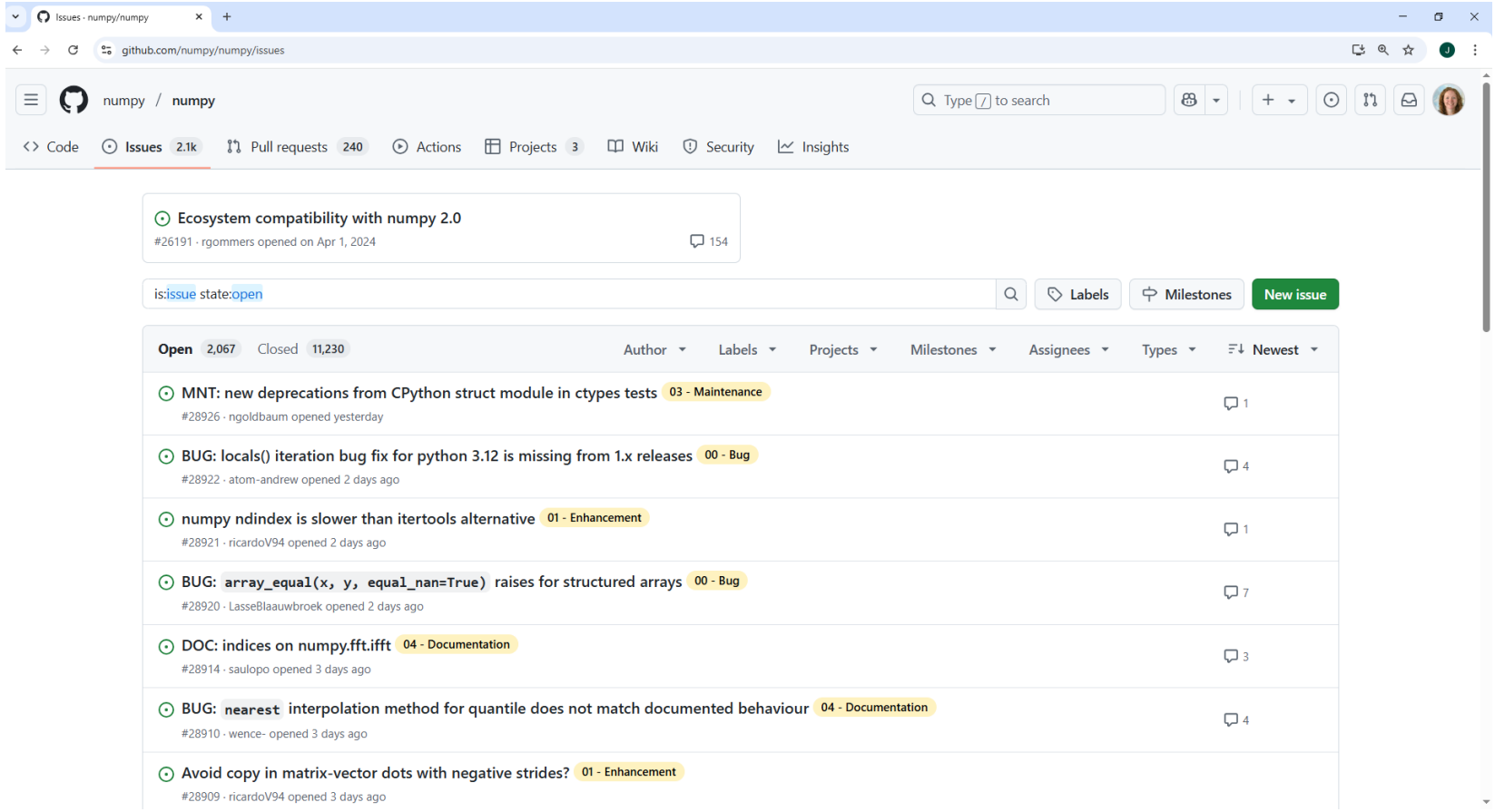
Can add multiple pages of documentation.

- Markdown formatting will be used, easy to edit with previews online.
- NB. Wiki files are not tracked in the core repository.
  - If prefer to edit locally can clone as separate repository: <repository-name>.wiki

- For standard user accounts, wiki pages are only available for public repositories.
- They are available for use in private repositories, under organisation (or paid) accounts.

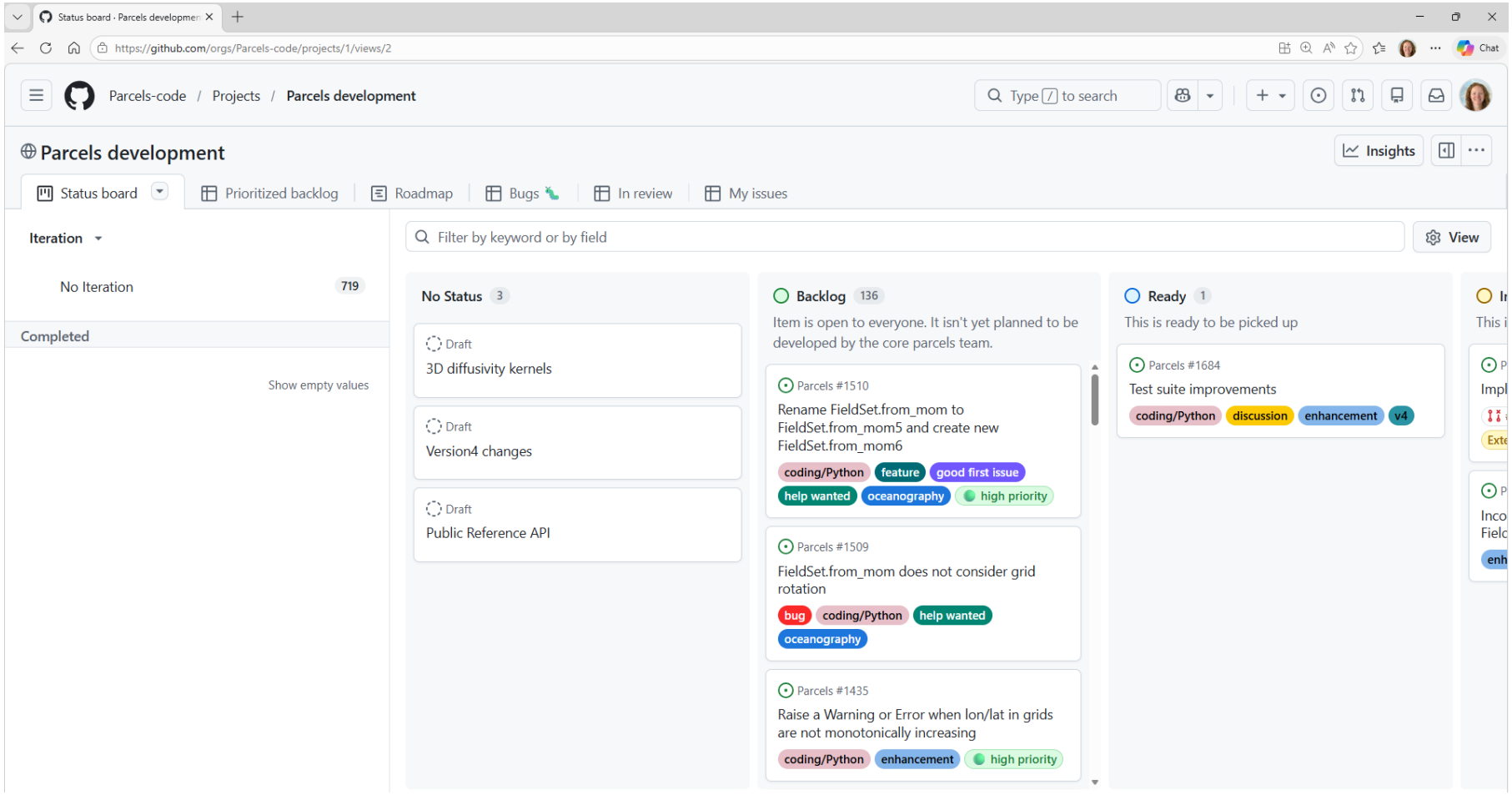
Issues

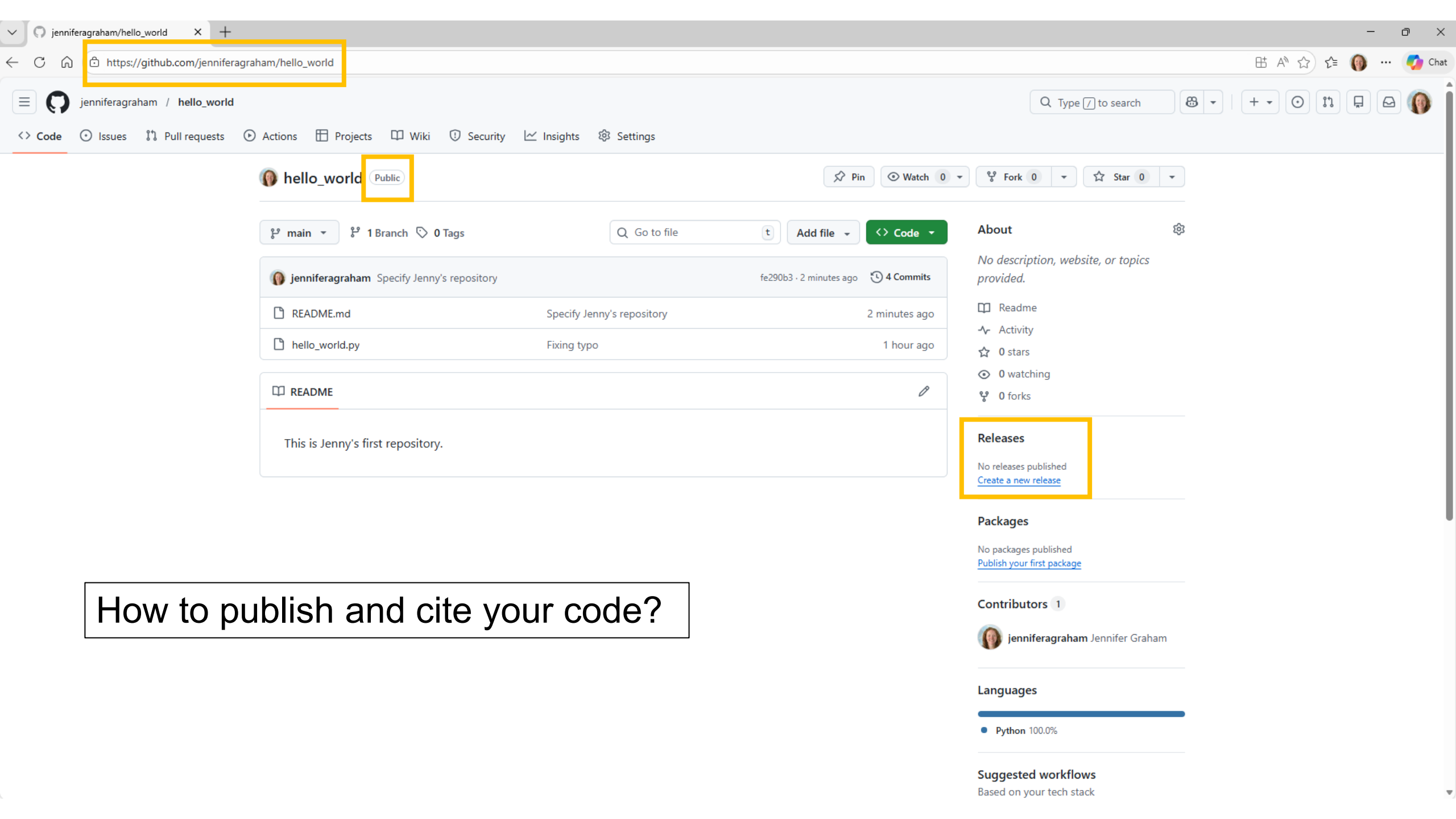
- A key feature for developers, useful to flag features, maintenance, or known problems/bugs.
- Issue numbers can be cited elsewhere e.g. in commit messages.



Projects

- Can be useful to outline planned development schedules for larger repositories, e.g., assigning work to different collaborators.




jenniferagraham/hello_world
https://github.com/jenniferagraham/hello_world
jenniferagraham / hello_world
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
hello_world
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
t
Add file
Code
jenniferagraham Specify Jenny's repository
fe290b3 · 2 minutes ago
4 Commits
README.md
Specify Jenny's repository
2 minutes ago
hello_world.py
Fixing typo
1 hour ago
README
This is Jenny's first repository.
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
jenniferagraham Jennifer Graham
Languages
Python 100.0%
Suggested workflows
Based on your tech stack
How to publish and cite your code?

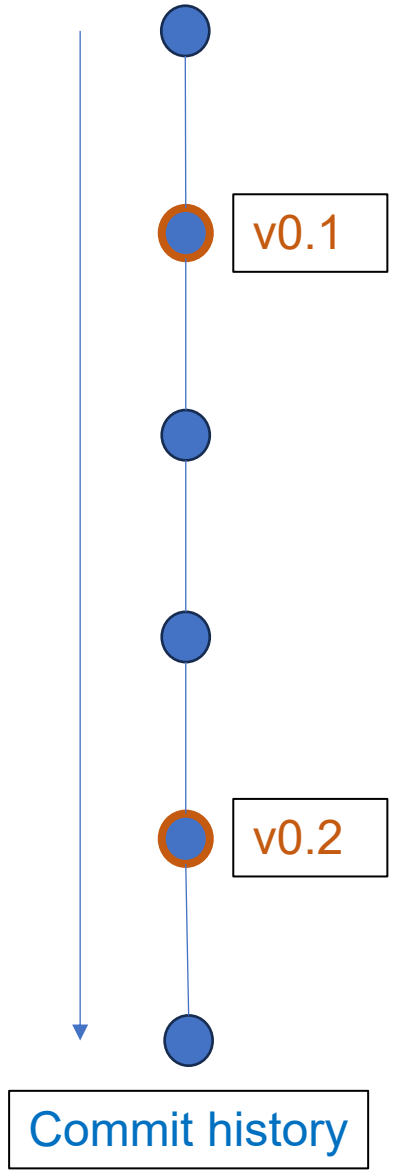
GitHub Releases

Creating a release ensures that you can refer to the code at your chosen point in time.

Each release is associated with a “tag” that refers to your chosen commit in the history of your repository.

Through Zenodo, you can also automatically assign a DOI to your release on creation.

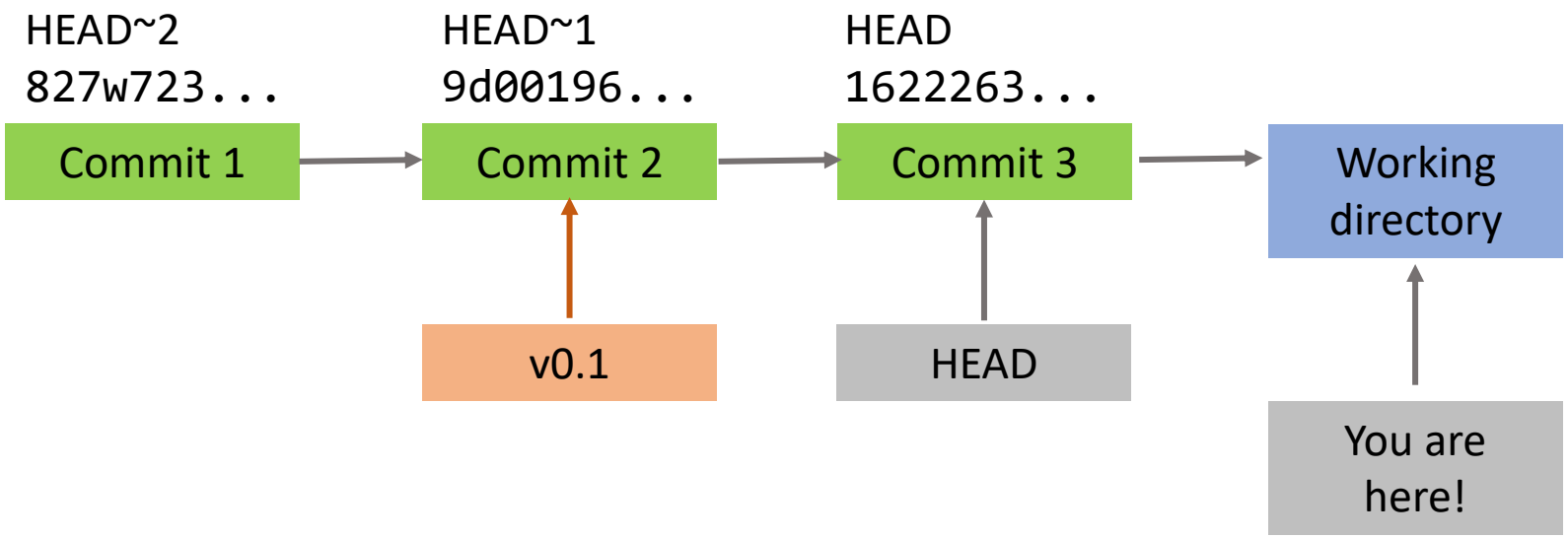
Advice on how to set this up here:

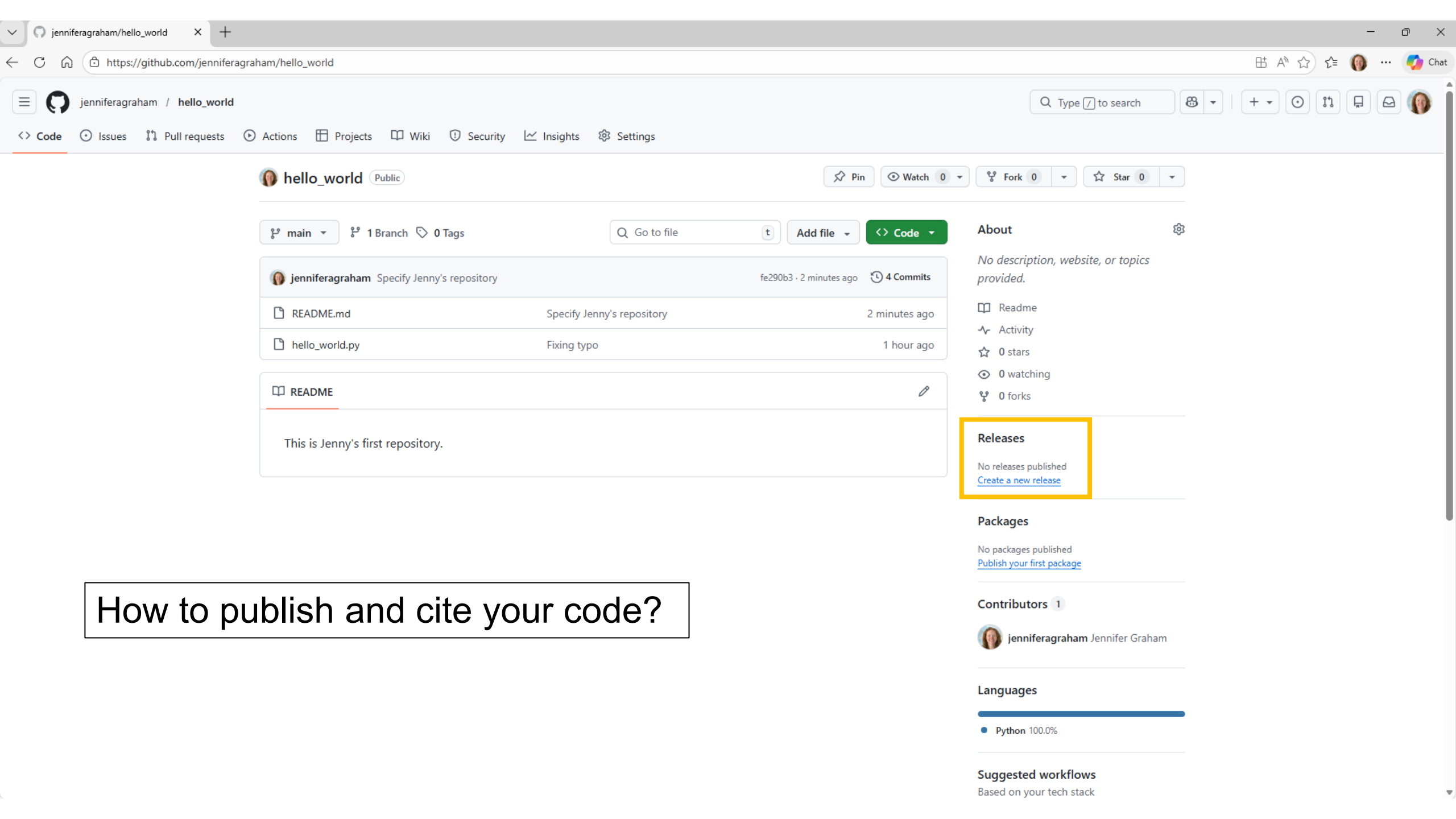


Git tag

Can also label commit versions on the command line, that you want to go back to, or release, in future.

```
git tag -a <tag-name> <commit> :: assign label <tag-name>
                                  to your commit
```





How to publish and cite your code?

← Releases

New release

Tag: v0.1 Target: main

Excellent! This tag will be created from the target when you publish this release.

Release title

First release

Release notes

Select a previous tag to create generated release notes

Previous tag: Auto Generate release notes

Write Preview

First release for testing.

Tagging suggestions

It's common practice to prefix your version names with the letter v. Some good tag names might be `v1.0.0` or `v2.3.4`.

If the tag isn't meant for production use, add a pre-release version after the version name. Some good pre-release versions might be `v0.2.0-alpha` or `v5.9-beta.3`.

Semantic versioning

If you're new to releasing software, we highly recommend to learn more about semantic versioning.

A newly published release will automatically be labeled as the latest release for this repository.

If 'Set as the latest release' is unchecked, the latest release will be determined by higher semantic version and creation date. Learn more about release settings.

Releases Tags

Draft a new release Find a release

1 hour ago

jenniferagraham

v0.1

2d39a54

Compare

First release

Pre-release

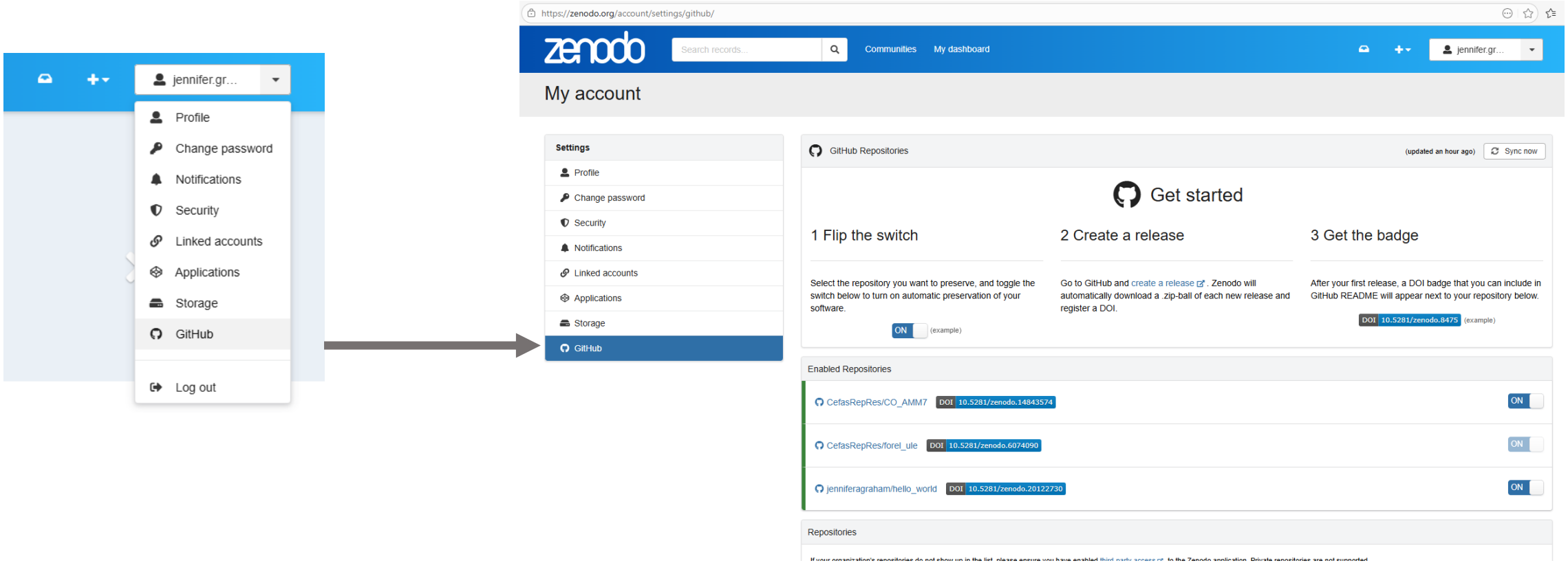
First release for testing.

https://doi.org/10.5281/zenodo.20122594

Assets 2

Creating a DOI for your release?

- Log in to Zenodo using your GitHub account, then “toggle” your repository.
  - When you sync your account, new releases will then be assigned a DOI.
  - NB. Only public repositories can be linked to Zenodo.



Try it yourself [20 min]

1. Document your code!
   - In your test repository (or one you made already), create a README file and/or wiki.
   - Use markdown formatting, e.g., to create headings and links to files or references.

2. Create a release of your repository.

   Optional:
   - *Create tag(s) for your chosen commit(s) on the command line.*
   - *Link your account to Zenodo, to create a DOI.*

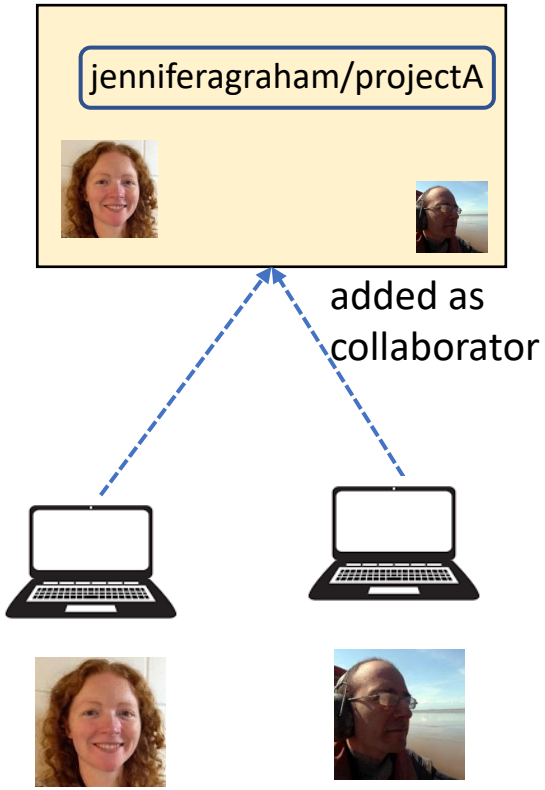
How to collaborate?

Sharing development of your code

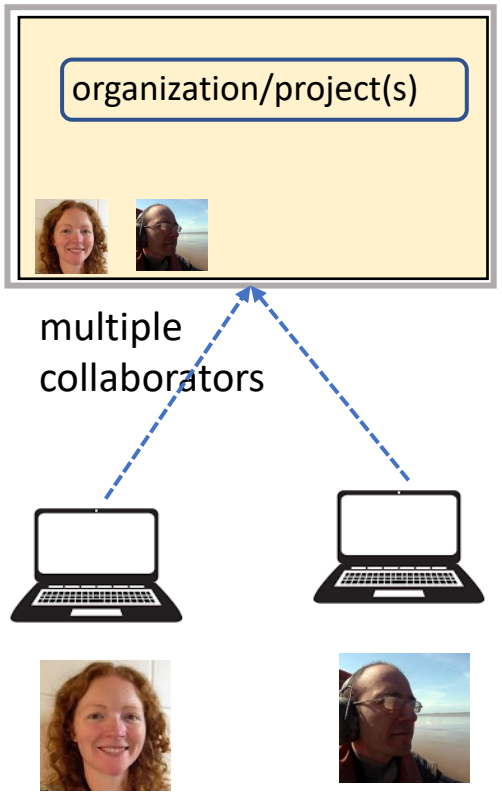
Collaboration models

GitHub

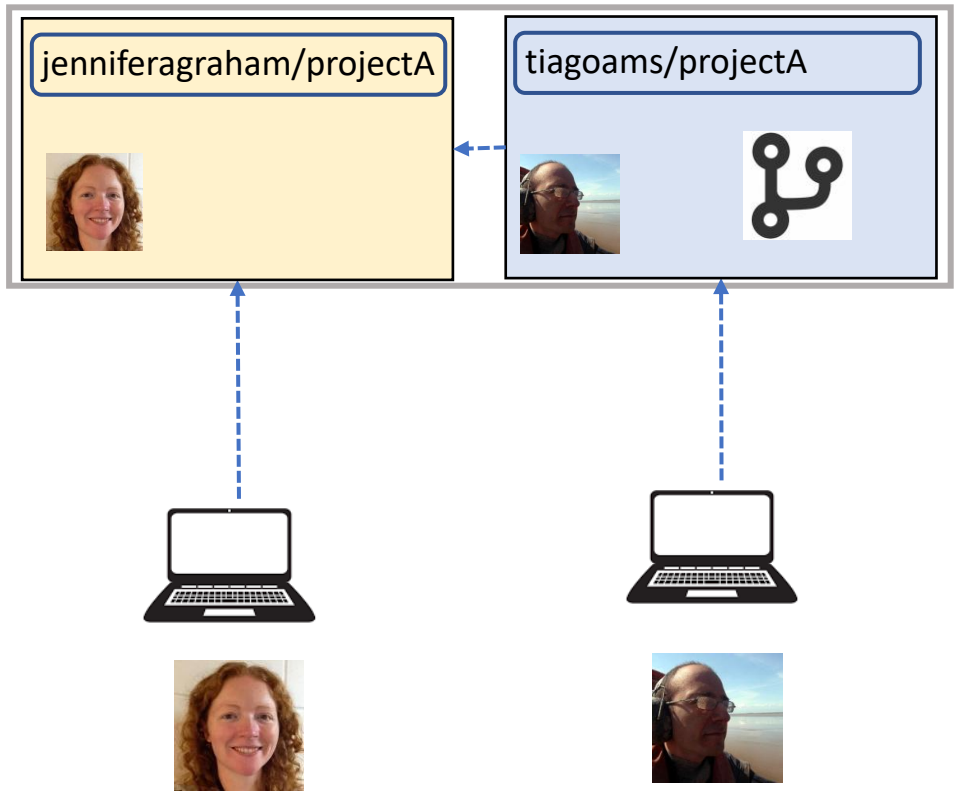
Personal account + add collaborator



GitHub Organization

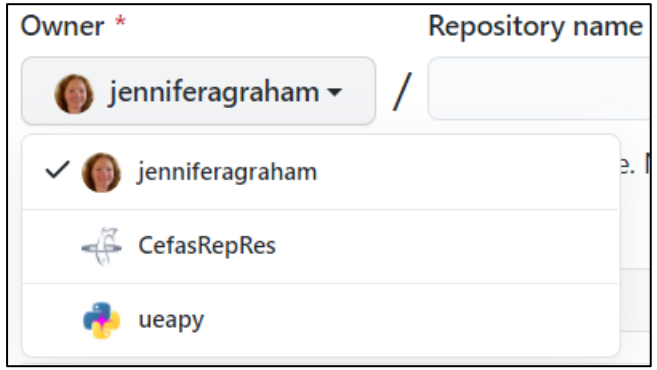


Forked + pull request



Organisations

- Organisations can add further functionality on GitHub.
  - An example from SAMS: ScotMarPhys
  - If you don't have one already, you can always create your own…
- Can develop private repositories that are still visible to all members.
- Easily share and organise work within the organisation.
- Repositories are technically "owned" by the organisation rather than person creating them.
  - NB. Select the correct "owner" on creation.



ScotMarPhys

Follow

Popular repositories

m_moorproc_toolbox Public

MATLAB based toolbox for real time and delayed mode processing of moored instrument data from OSNAP and CLASS oceanographic cruises.

MATLAB ☆ 2

glider_toolbox Public

Forked from socib/glider_toolbox

MATLAB/Octave scripts to manage data collected by a glider fleet, including data download, data processing and product and figure generation, both in real time and delayed time.

MATLAB

ACExR_fjord_model Public

MATLAB

ScotMarPhys.OSNAP-Mooring-Processing.io Public

HTML

ocean-glider-processing Public

Jupyter Notebook

ecoSUB-toolbox Public

A toolbox for processing ecoSUB data files, including basic quality control, data conversion, and plotting of data.

Jupyter Notebook 1

People

This organization has no public members. You must be a member to see who's a part of this organization.

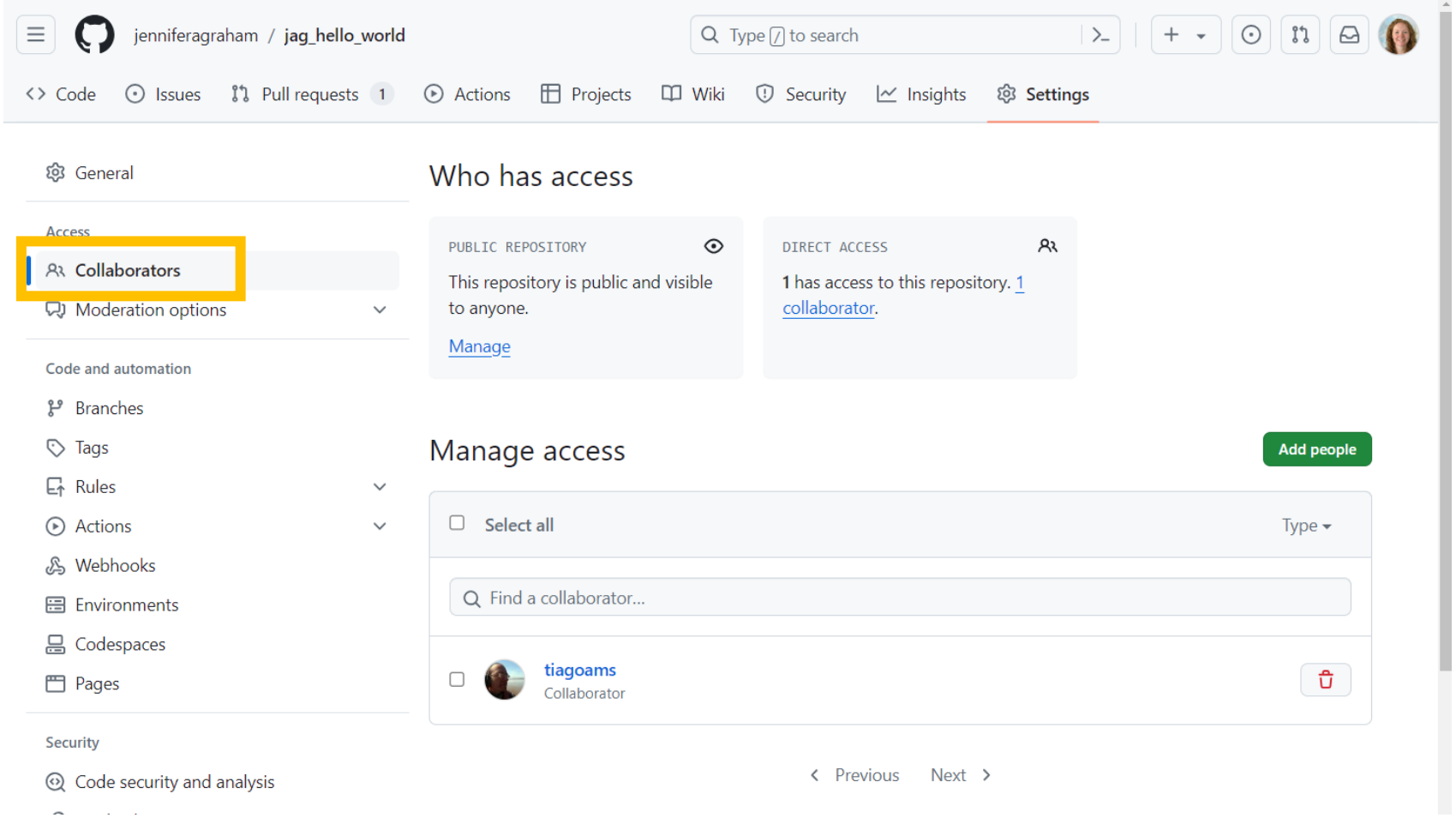
Top languages

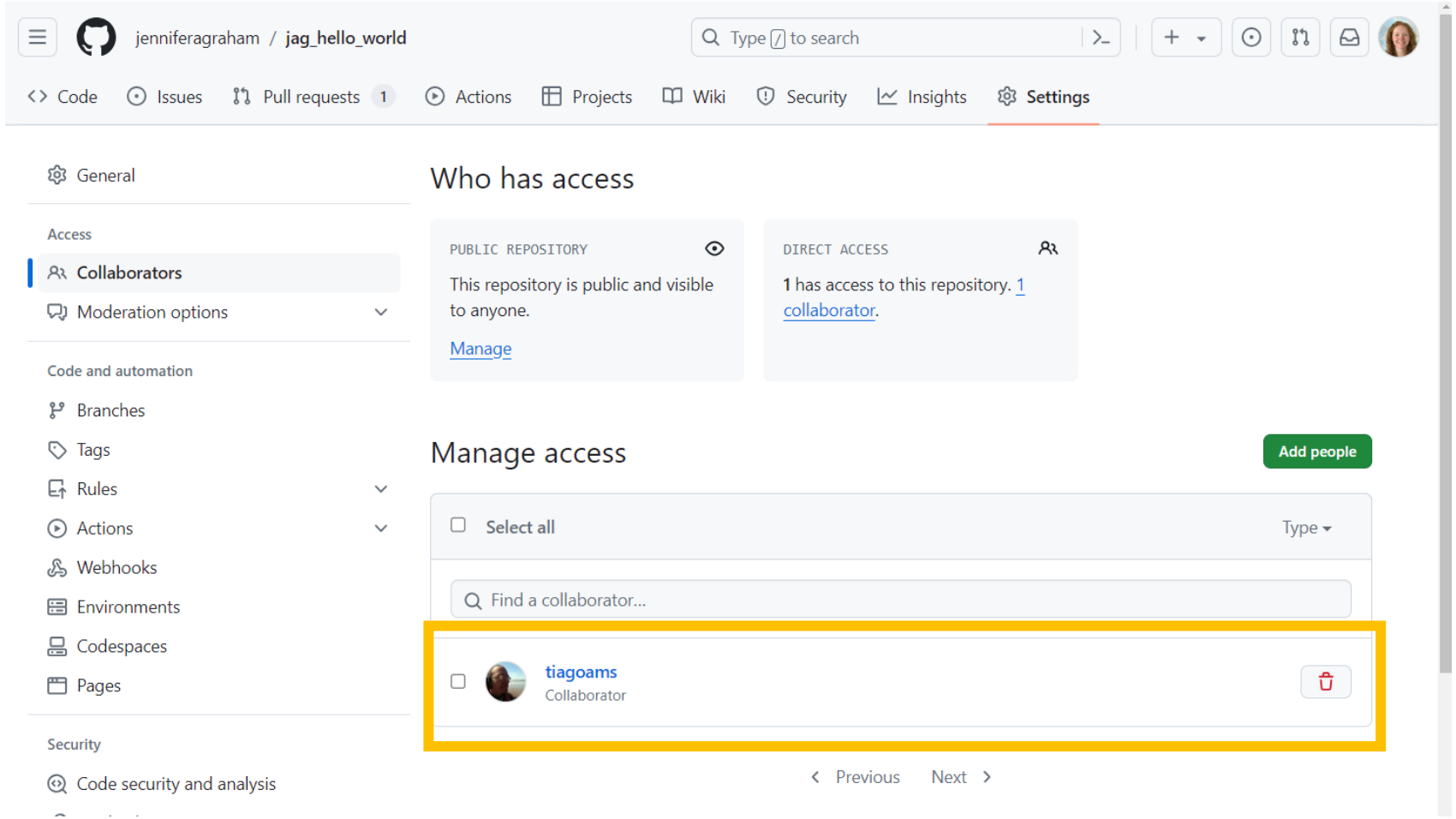
MATLAB Jupyter Notebook HTML

Report abuse

Controlling repository access…

- Collaborators can be added to each repository.





- For **personal** repositories, there are two levels of permission:
  - **Repository owners** (Admin permissions)
  - **Collaborators** (aka Write access etc.)

- Only the owner can invite new collaborators.

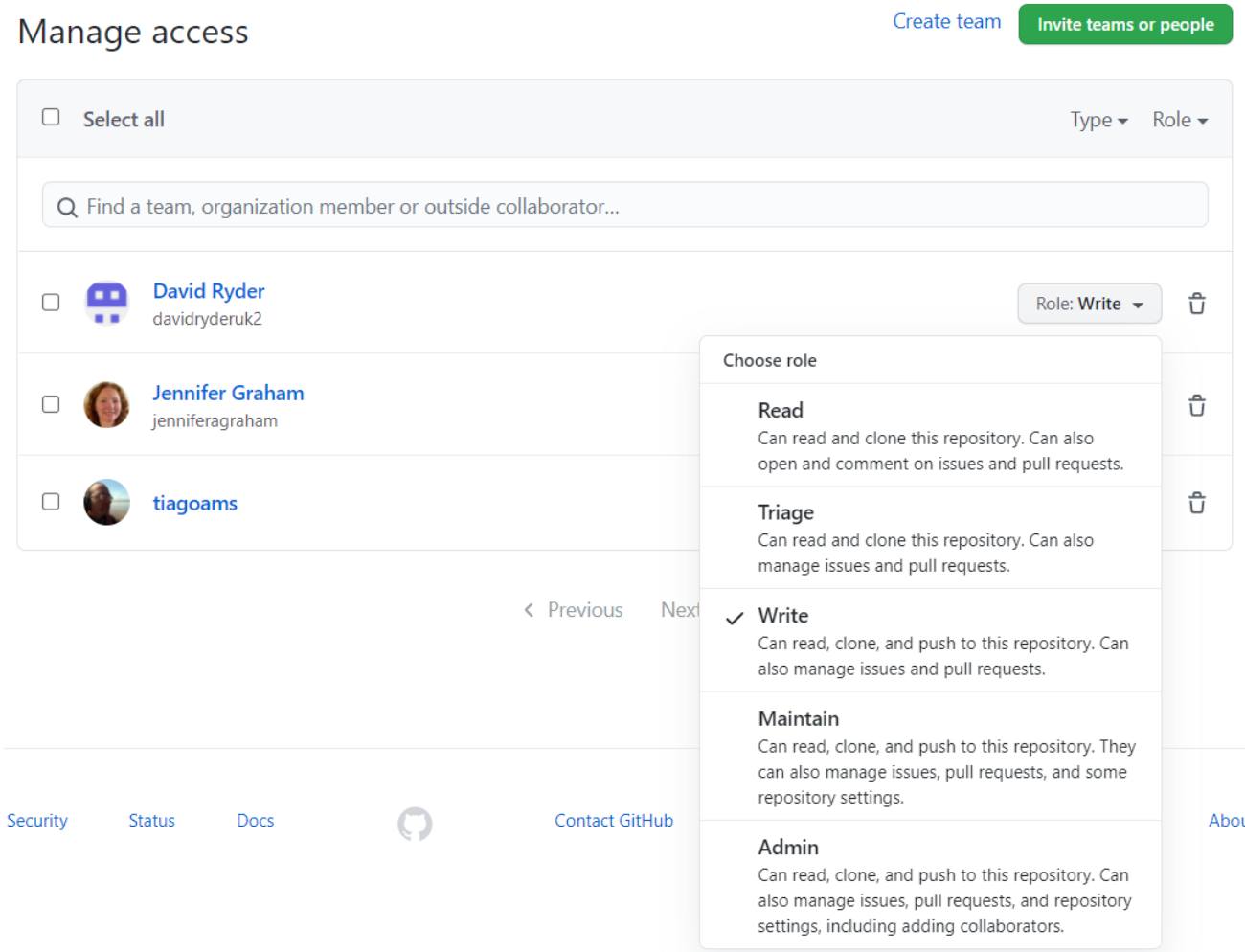
- For **organisation** repositories, there are many levels available:

Role	Access
Admin	Full access i.e. repository owners
Maintain	Manage the repository with access to all but most sensitive settings.
Write	Can push changes to repo
Triage	Can manage issues but no write access.
Read	View code, make comments, but not change

- Only those with **Admin** access can invite new collaborators.

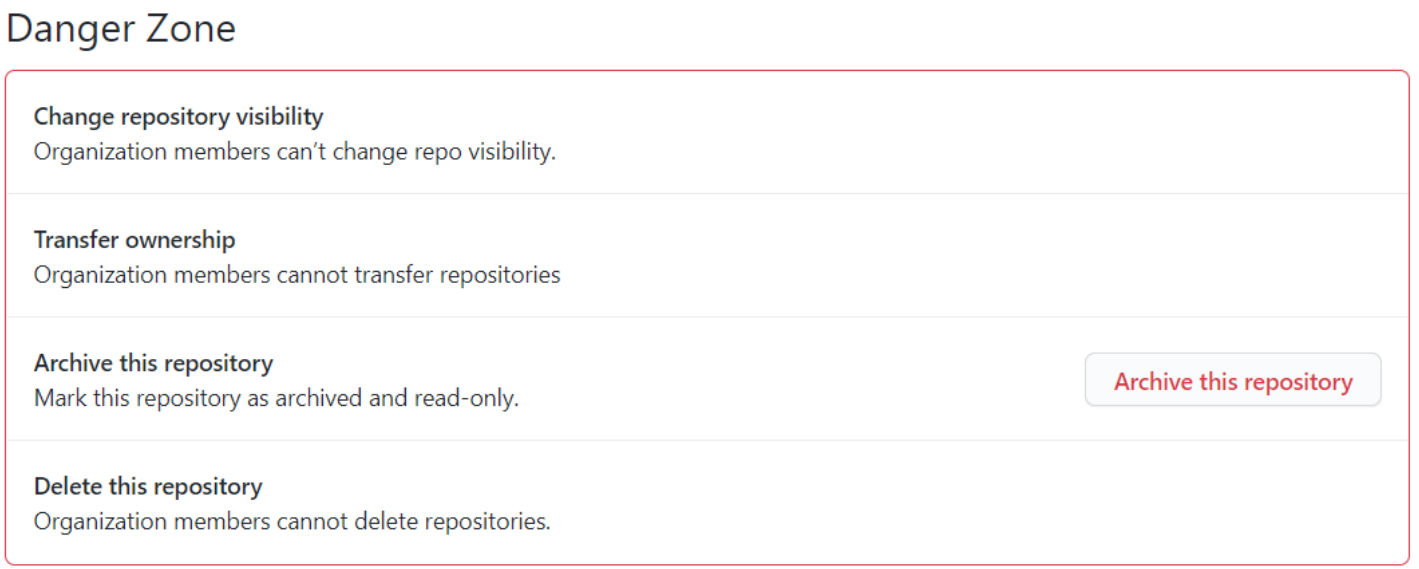
- Access can be grouped via “teams” as well as individuals, e.g., if regular groups require access to multiple repositories.

- NB. Outside collaborators can be invited (don’t need to be a member to collaborate)



Restricted admin access

- Within an **organisation**, repositories are technically owned by the organisation, not individual.
  - Some settings can only be changed/managed by the **organisation owner** e.g.



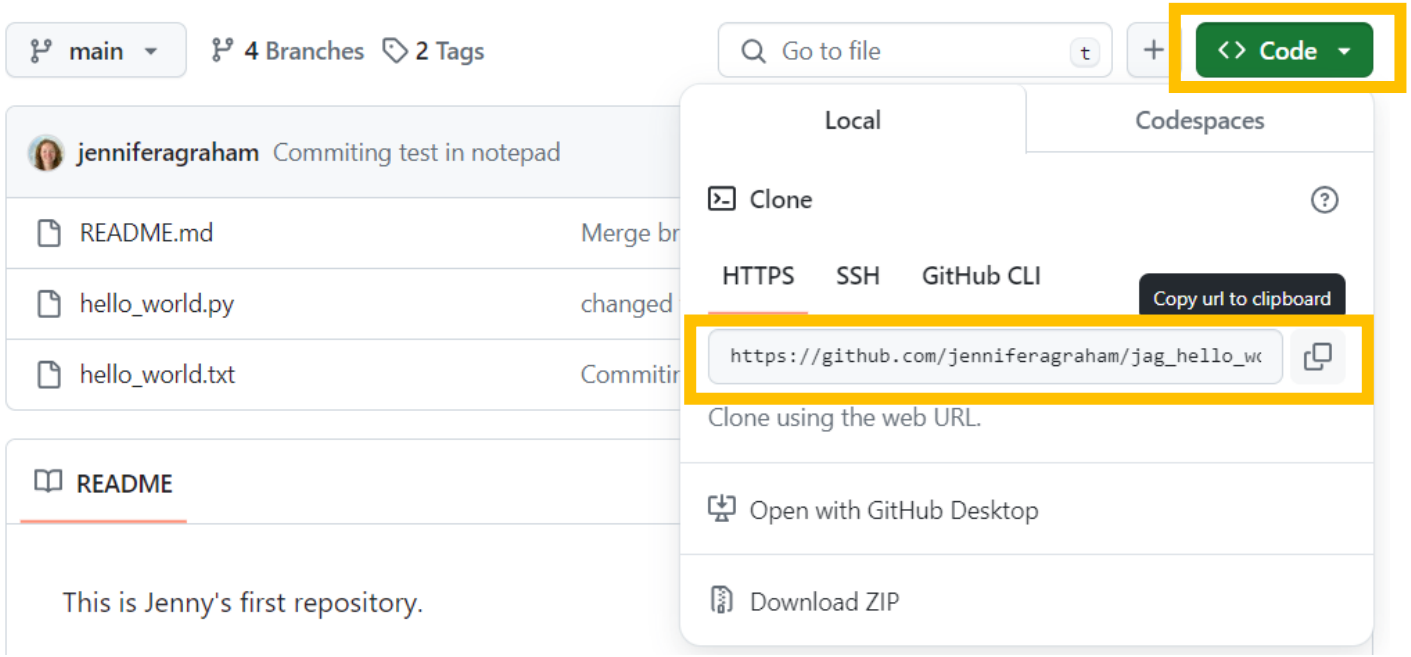
- Organisation owners can decide on the level of access for members.
- Only owners of the organisation can invite/approve new members.

(How) can I modify someone else's repository?

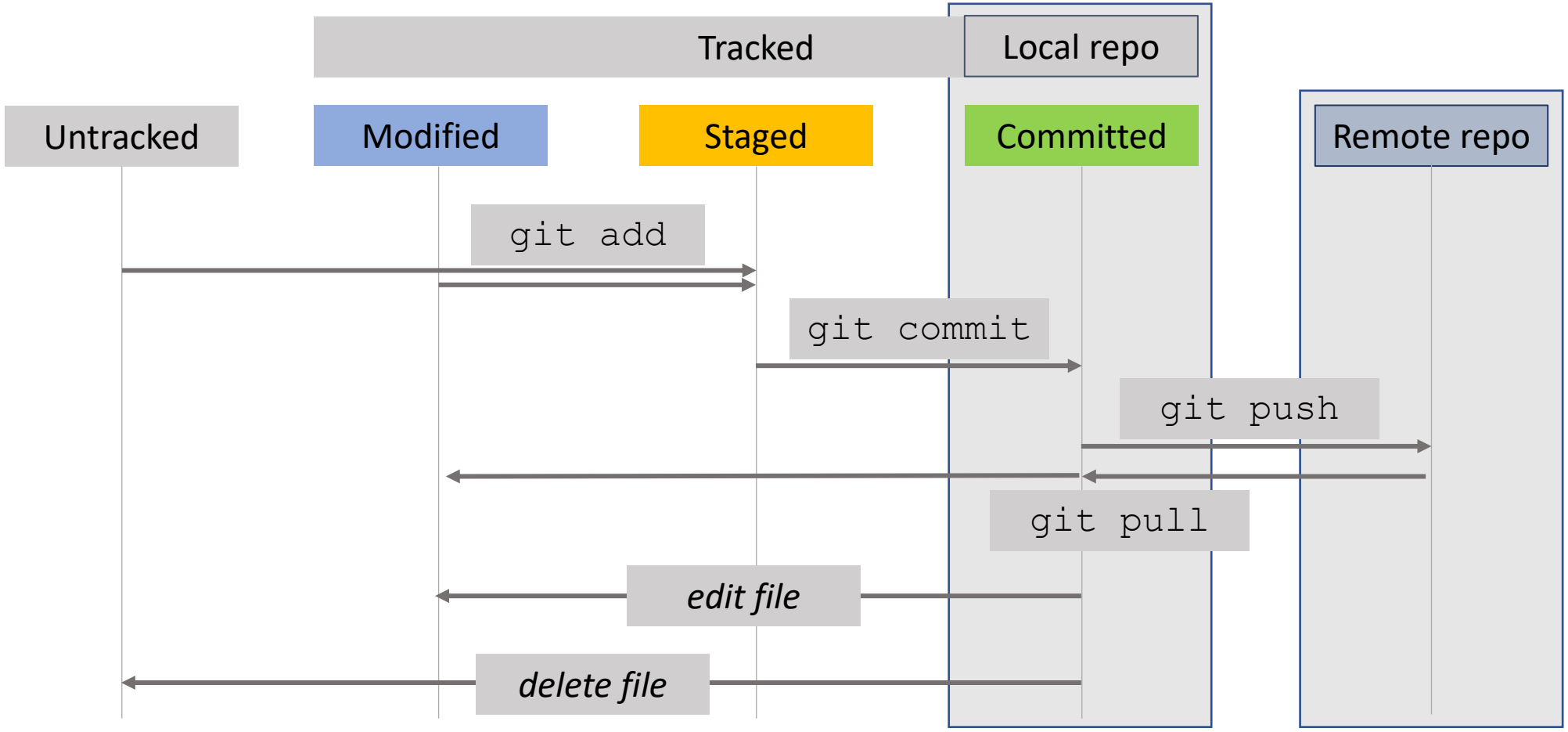
It depends on your access…

Admin, write or collaborator access…

- Use the same workflow discussed already.
  - You can push changes to main, and create branches for developments.
- Use `git clone` to make a local copy.




```
git clone <copied-url>

download into new local repository
```


Tracked
Local repo
Untracked
Modified
Staged
Committed
Remote repo
git add
git commit
git push
git pull
edit file
delete file

If you have only read access…

e.g. For public repositories, or those shared through an organisation.

- You can still clone the repository, then track locally as normal.
- However, if you modify the repository, you won't be able to push changes back to the remote repository.

- If you want to make a copy and intend to make developments, consider forking…

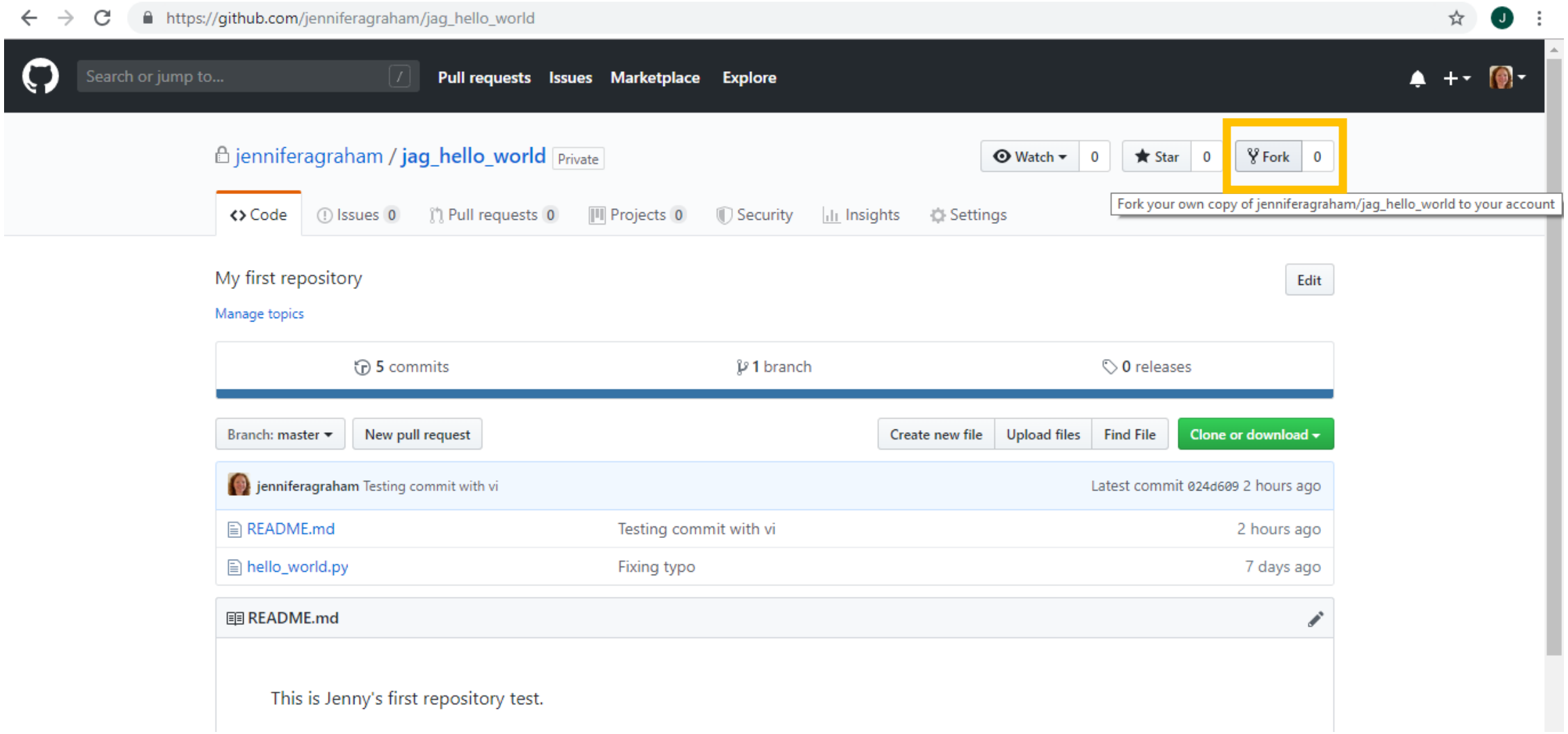
What is a fork?

- Similar to a branch, in that both can be used to develop new features, away from the main branch.
- In basic terms, the difference is:
  - If you have write access for the repository -> branch.
  - If you don’t have write access -> fork.

- Forking creates a copy of the original repository in your ownership (including its history).
- Repository remains linked to the original (main) so you can see how the developments diverge.
- From a fork, you must create a “pull request” to merge changes back to the original.
  - This will notify the code owner, and allow them to review changes before deciding whether or not to accept/merge (i.e. request them to pull changes into their repository).

How to create a fork?

Note: you can't create a fork owned by the same individual or organisation as the original.



https://github.com/CefasRepRes/jag_hello_world

Search or jump to... / Pull requests Issues Marketplace Explore

CefasRepRes / jag_hello_world Private
forked from jenniferagraham/jag_hello_world

Watch 0 Star 0 Fork 1

Code Pull requests 0 Projects 0 Security Insights Settings

My first repository

Edit

Manage topics

5 commits 1 branch 0 releases

Branch: master New pull request Create new file Upload files Find File Clone or download

This branch is even with jenniferagraham:master. Pull request Compare

jenniferagraham Testing commit with vi Latest commit 024d609 19 hours ago

README.md	Testing commit with vi	19 hours ago
hello_world.py	Fixing typo	8 days ago

README.md

This is Jenny's first repository test.

https://github.com/CefasRepRes/jag_hello_world

Search or jump to... / Pull requests Issues Marketplace Explore

CefasRepRes / jag_hello_world Private
forked from jenniferagraham/jag_hello_world

Watch 0 Star 0 Fork 1

Code Pull requests 0 Projects 0 Security Insights Settings

My first repository

Edit

Manage topics

5 commits 1 branch 0 releases

Branch: master New pull request Create new file Upload files Find File Clone or download

This branch is 2 commits behind jenniferagraham:master. Pull request Compare

jenniferagraham Testing commit with vi Latest commit 024d609 19 hours ago

README.md	Testing commit with vi	19 hours ago
hello_world.py	Fixing typo	8 days ago

README.md

This is Jenny's first repository test.

https://github.com/jenniferagraham/jag_hello_world/network/members

Search or jump to... / Pull requests Issues Marketplace Explore

jenniferagraham / jag_hello_world Private

Watch 0 Star 0 Fork 1

Code Issues 0 Pull requests 0 Projects 0 Security Insights Settings

- Pulse
- Contributors
- Traffic
- Commits
- Code frequency
- Dependency graph
- Network
- **Forks**

jenniferagraham / jag_hello_world
CefasRepRes / jag_hello_world

Note: You can't create a private fork, it will always be visible to the original owner.

Branch	Fork
• Created in your repository • You have total control over what you merge • Pull requests are sent to your own repository • Always intended to merge successful branches into master	• Creates an independent copy of another person's repository on your account so you can make changes • You can suggest changes to the original project but they don't have to merge them • Pull requests automatically go back to the original project • Allows you to take a GitHub project in a different direction - may never intend to merge with original project

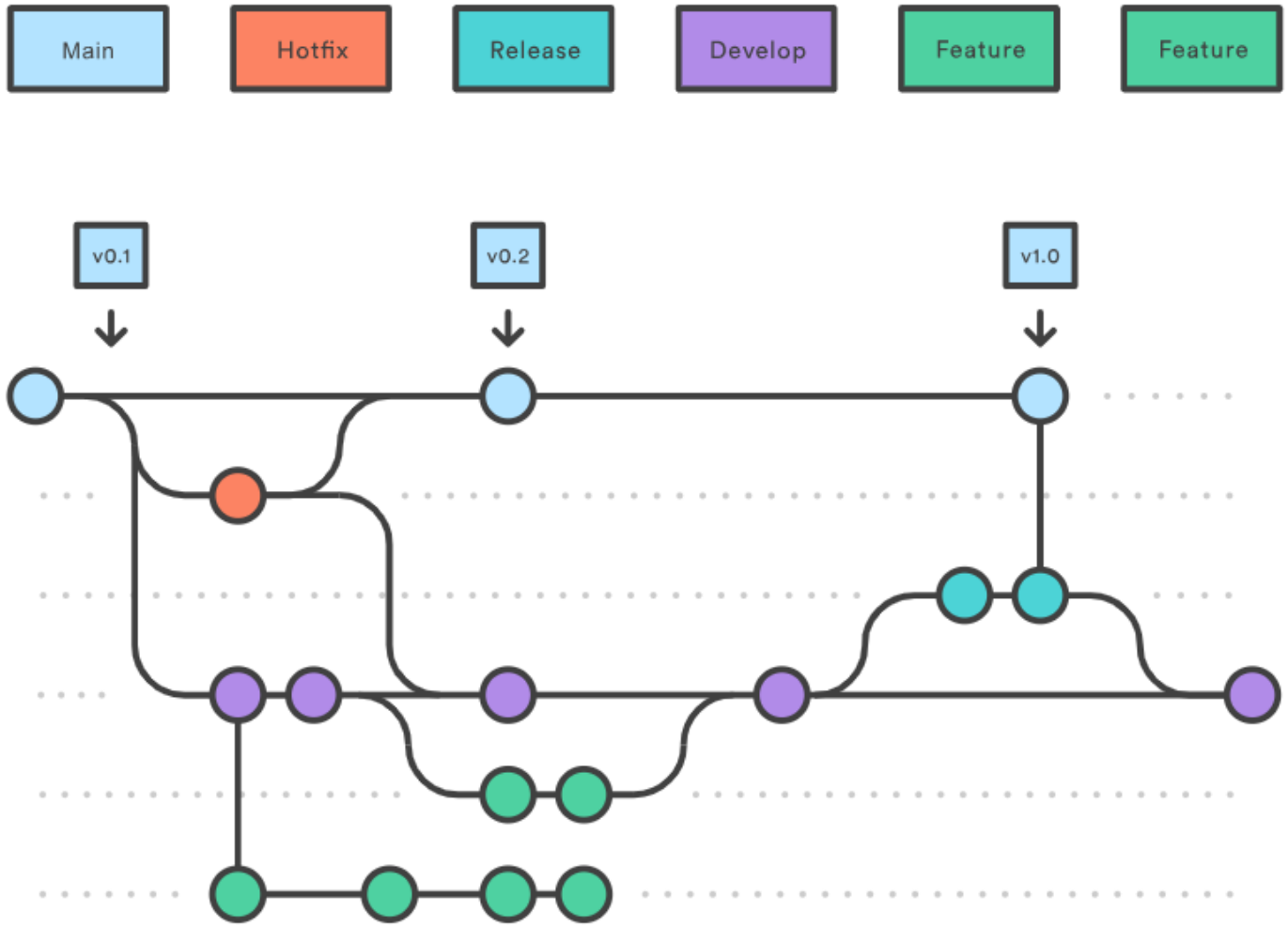
Collaboration workflows…

Collaboration workflows…

- git is just the tool

- Many workflows can be used, making use of various features available, and will depend on:
  - number of collaborators
  - frequency code is written
  - type of organisation (community, corporation)
  - criticality of the code

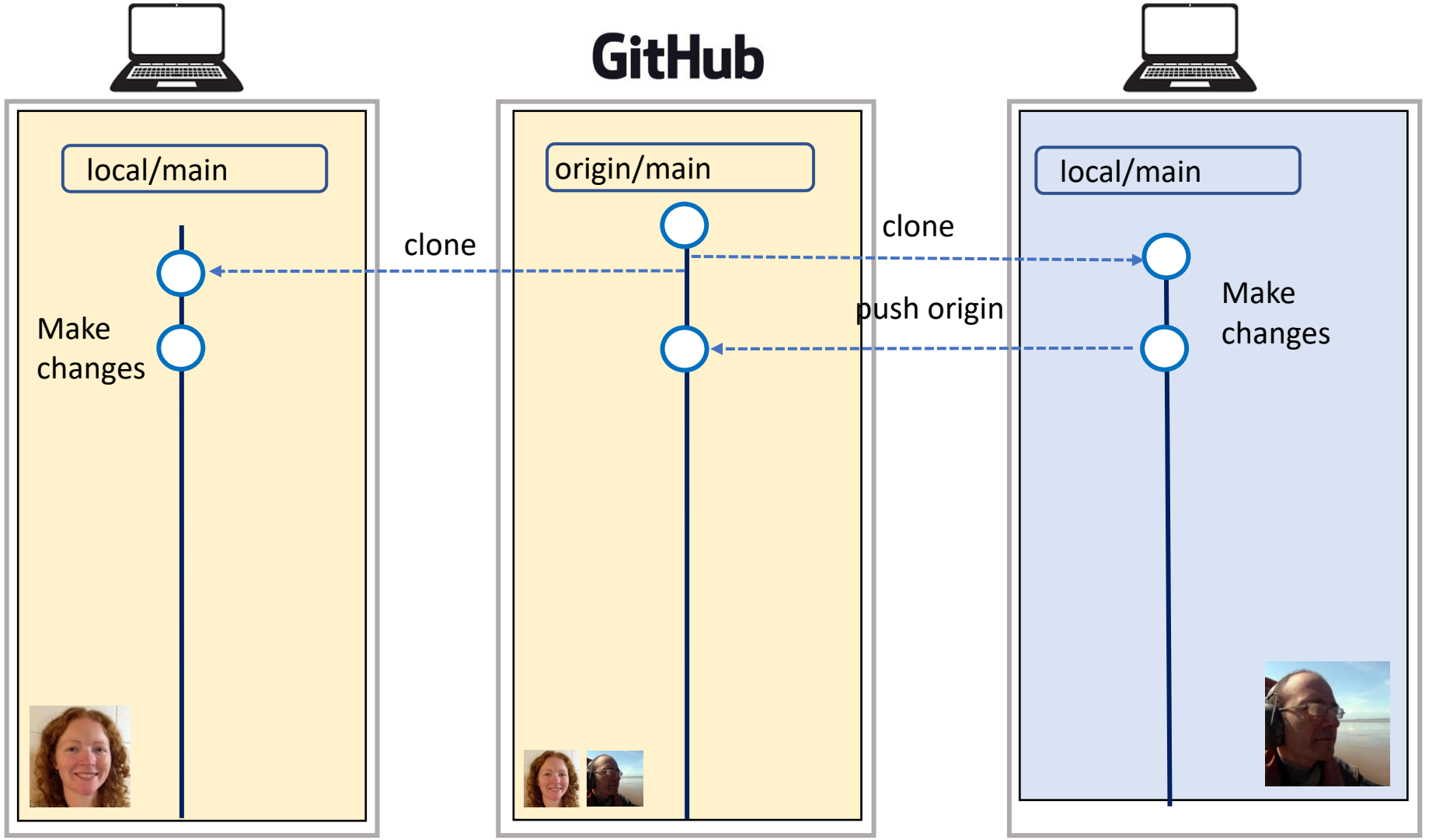
Shared access workflow, e.g., Gitflow



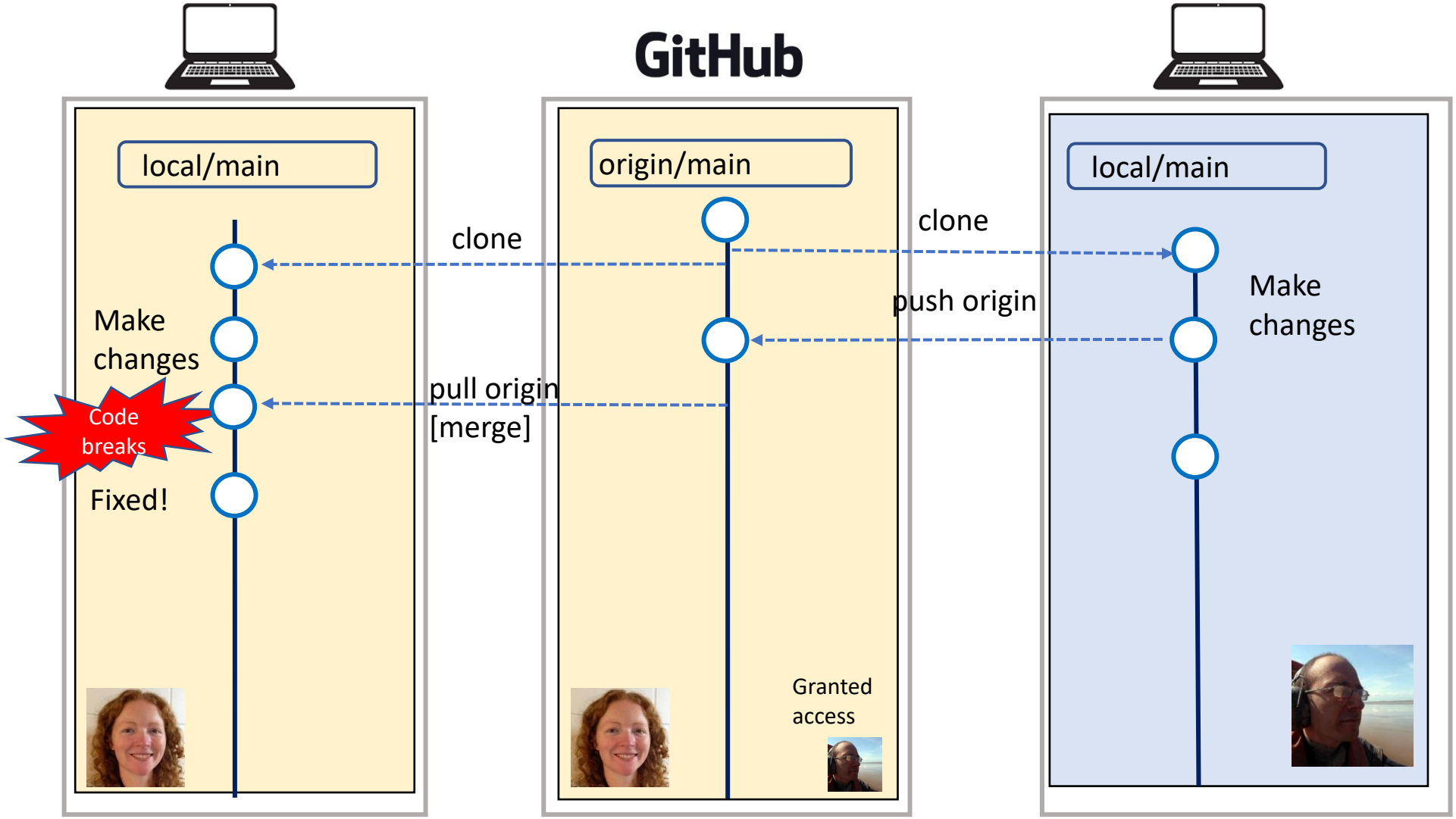
Many software developers use strict branching systems to ensure “the main is always shippable”.

Features always tested on development branch before release, so they don’t break the code!

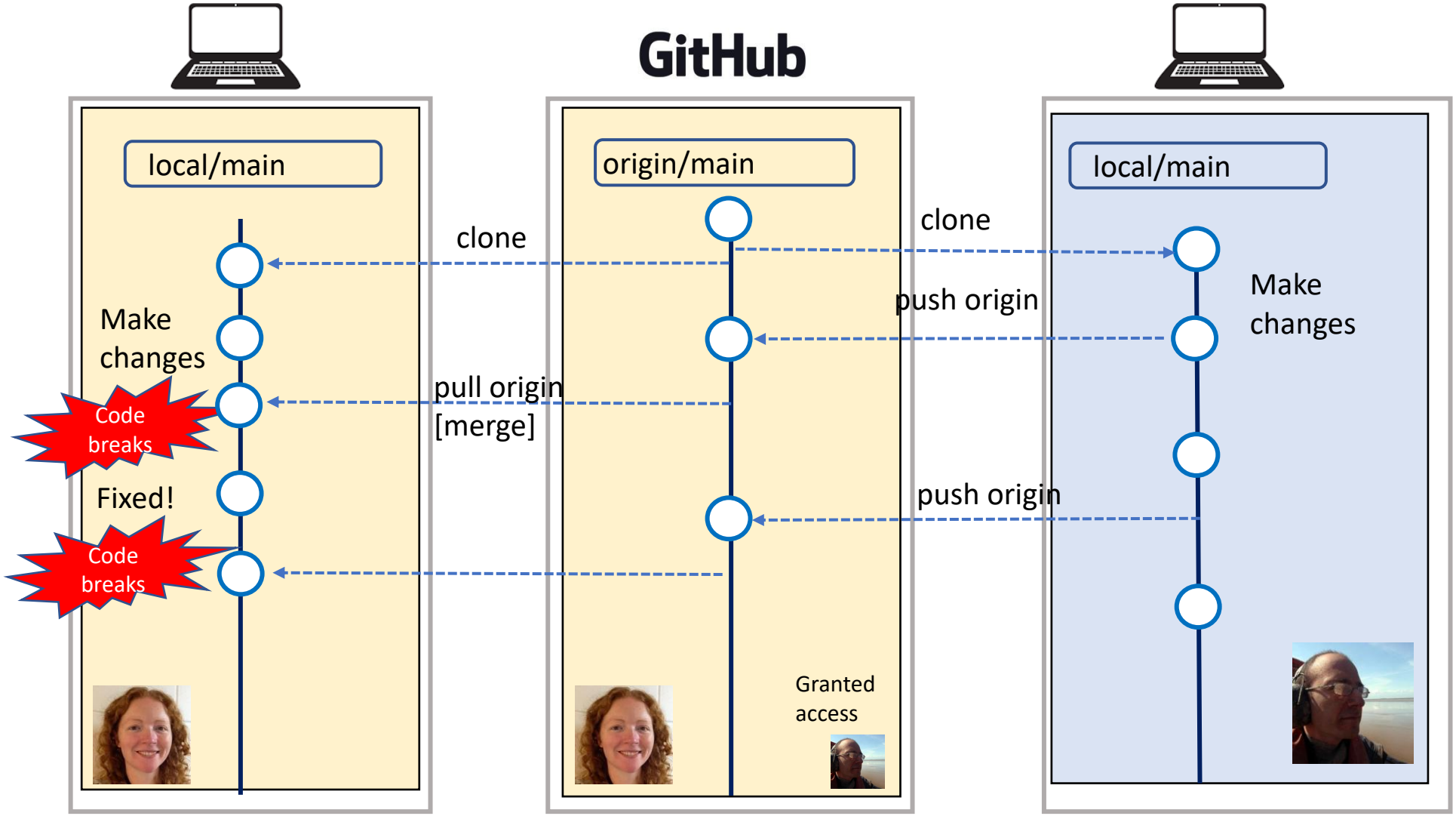
Bad workflow: competitors vs. collaborators



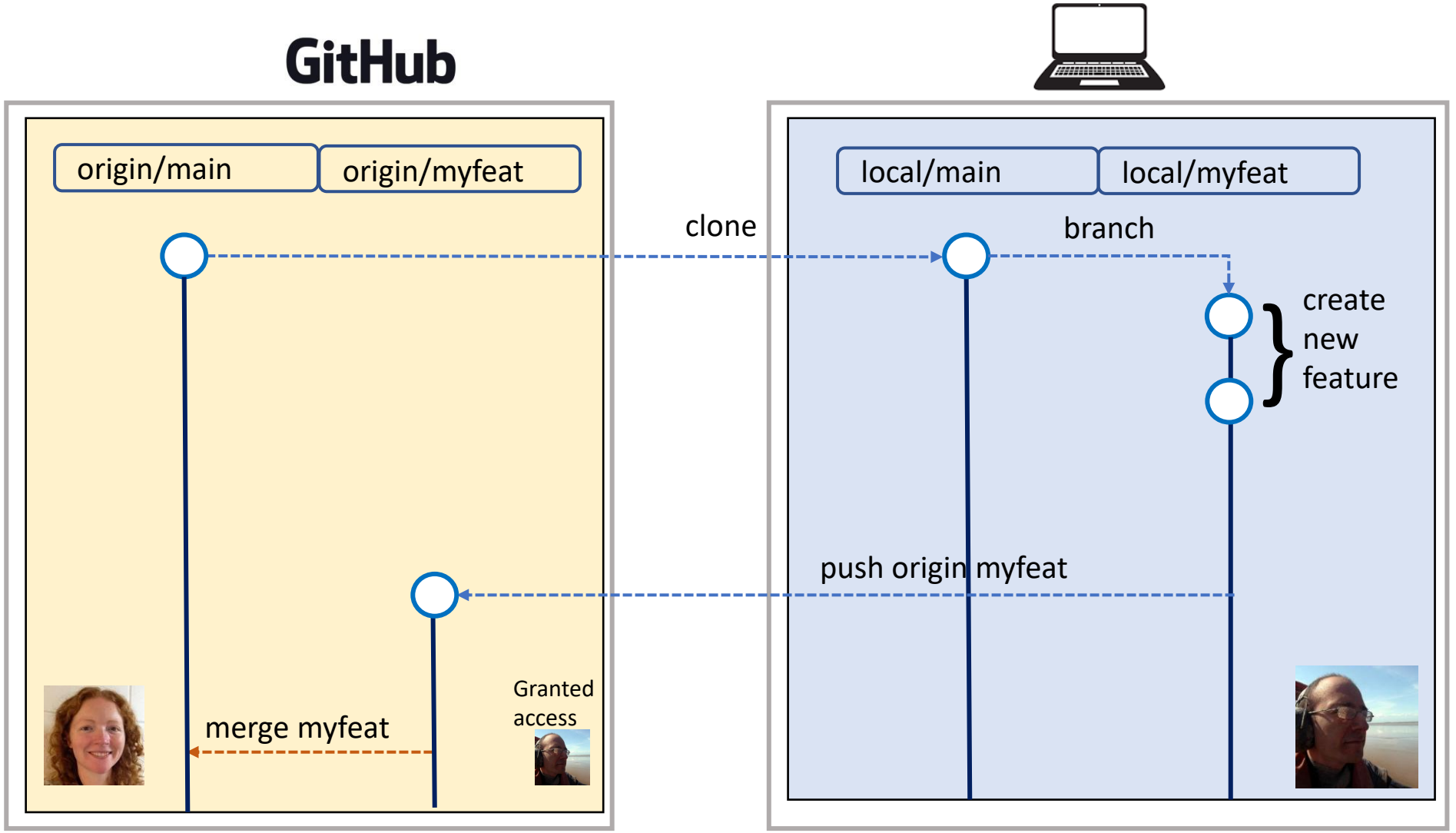
Bad workflow: competitors vs. collaborators



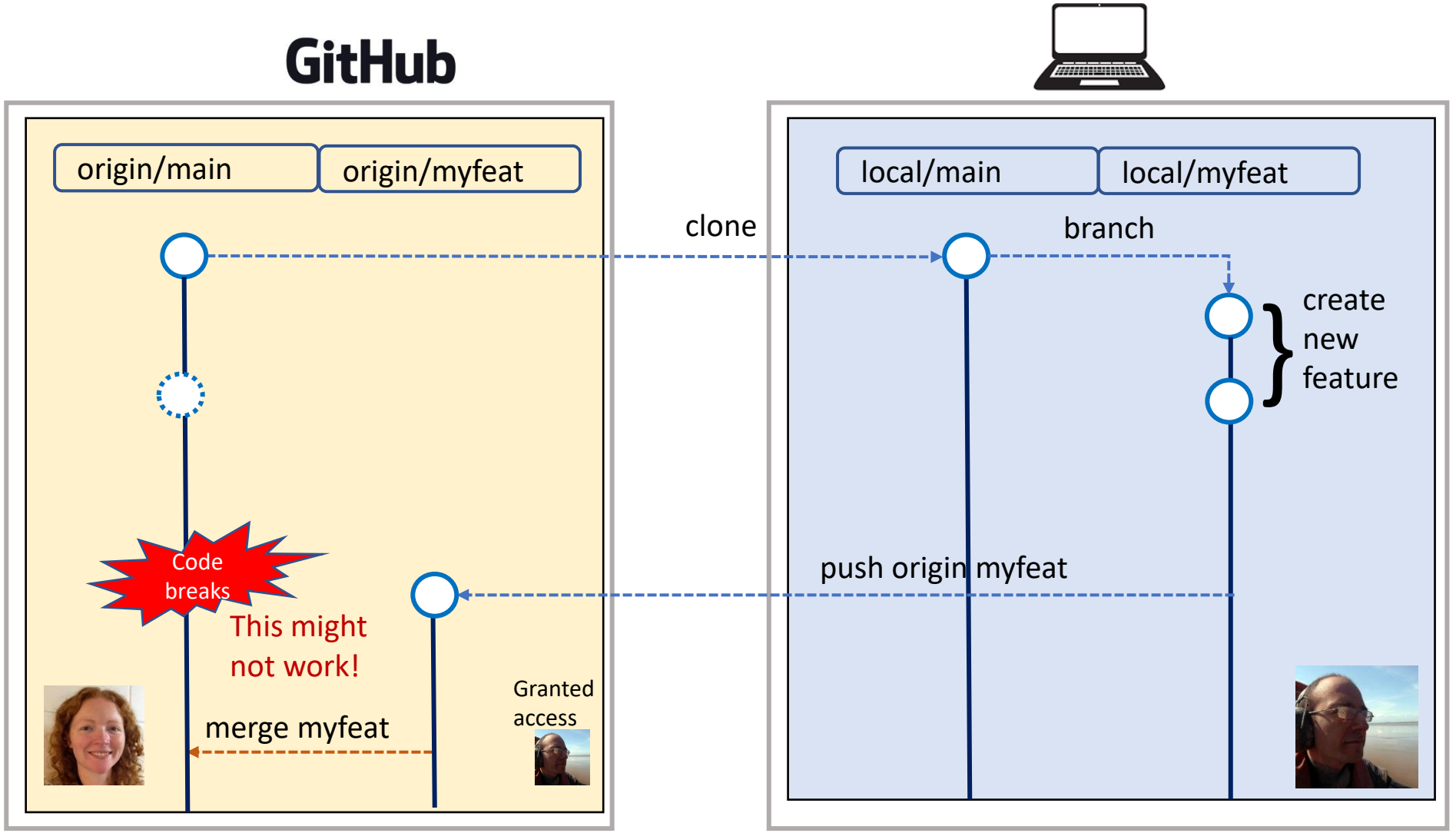
Bad workflow: competitors vs. collaborators



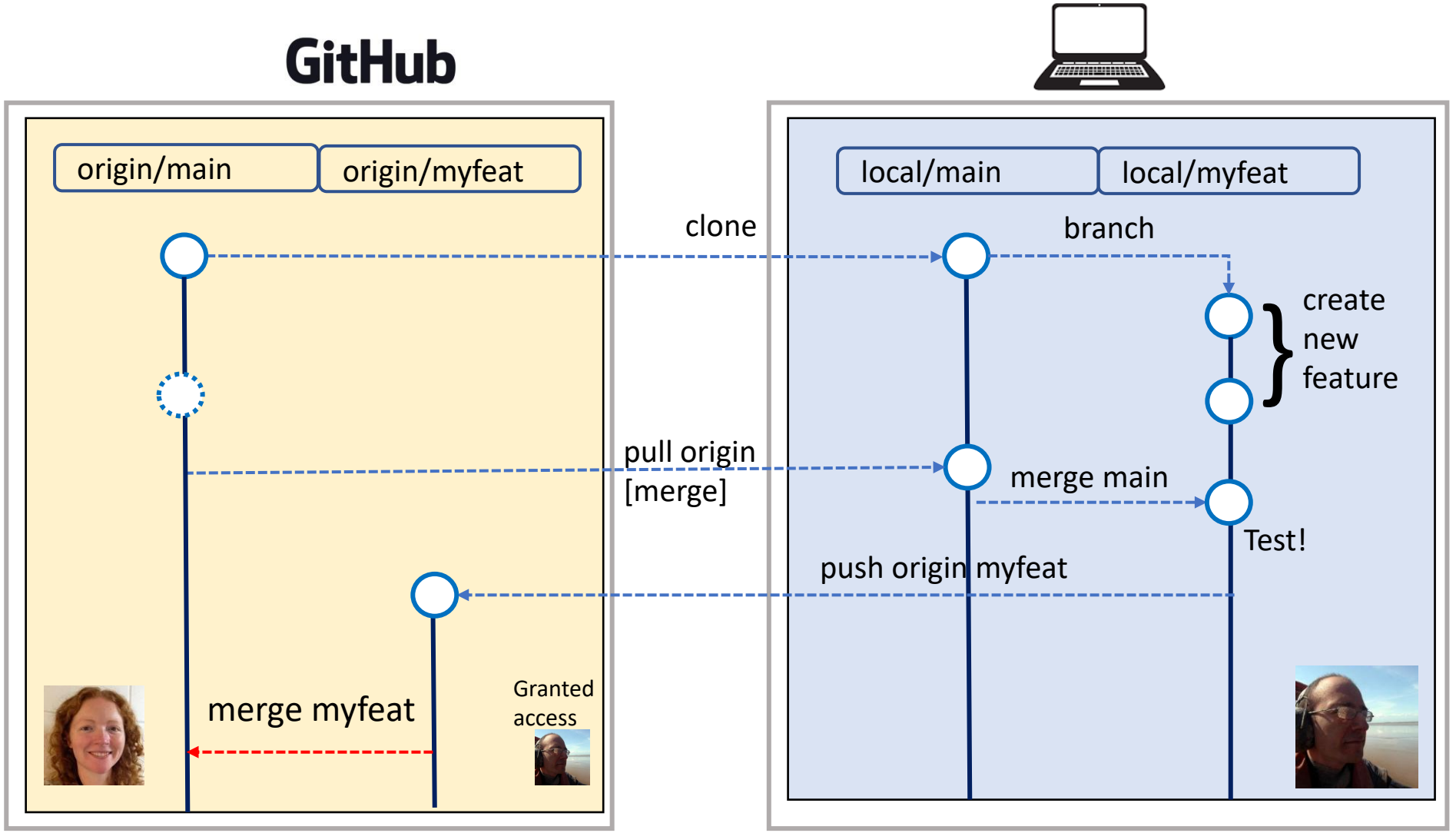
Shared workflow: Feature branches



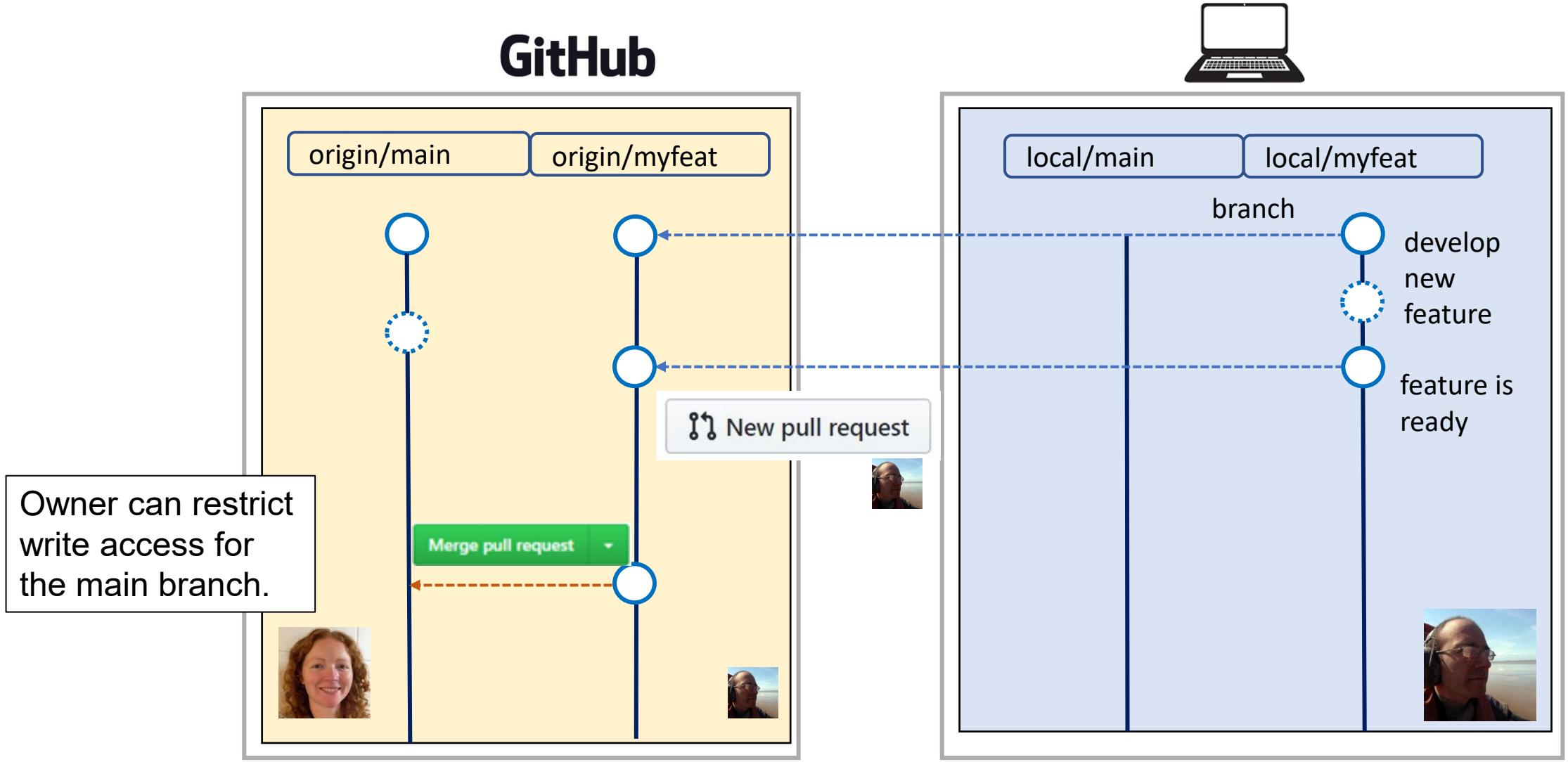
What if main branch has changed?



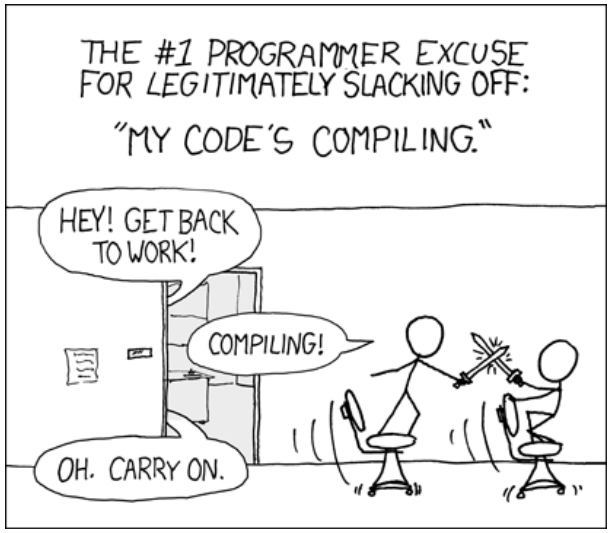
What if main branch has changed?



Alternative option: create a pull request



Try it yourselves [20 min]



Share your repositories!

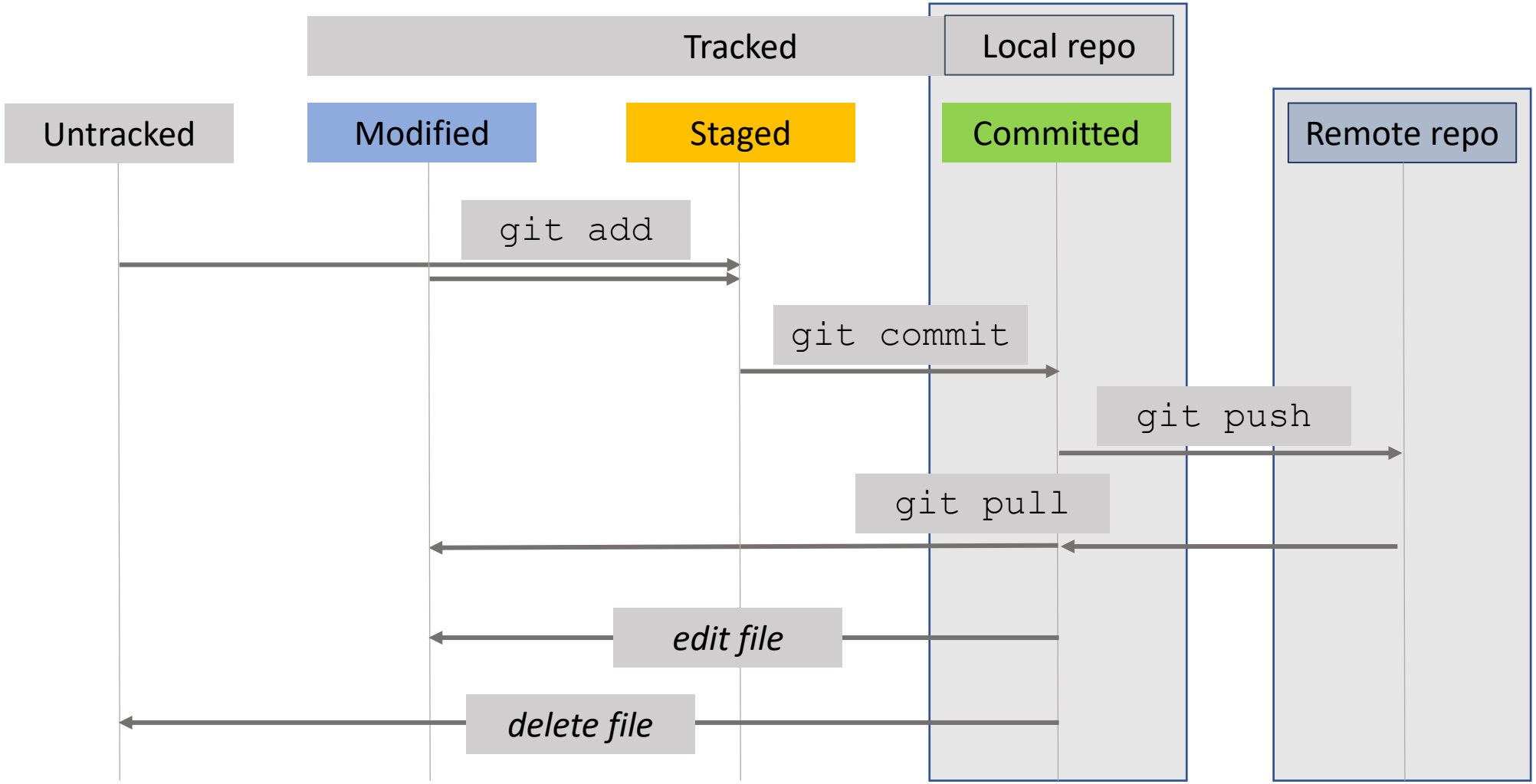
- Pair up and find each other's test repositories.

Choose how you want to collaborate?

1. Provide **write access** to colleague on test repository
   - Clone, (create branches?) make commits and push changes.

   Or
2. **Create a fork** and suggest changes using pull requests.
   - Clone, (create branches?) make commits and push changes
   - Create the pull request on GitHub.
   - Let the owner review changes.

When you start collaborating, check if there are any conflicts to be dealt with…?


Tracked
Local repo
Untracked
Modified
Staged
Committed
Remote repo
git add
git commit
git push
git pull
edit file
delete file

Please provide feedback:

All comments will be anonymous